



National University  
Manila

College of Computing and Information Technologies  
Computer Science Department

Development of a Proposed Facebook Content Management System using JavaFX and  
MongoDB

Final Project Requirement in Advanced Database Systems

Project Members

FORYASEN, Blix D.  
LIBUNAO, Trixia Nicole A.  
NATIVIDAD, Franze Carlo J.

Professor

Armida P. Salazar

June 27, 2024

## Table of Contents

|                                              |           |
|----------------------------------------------|-----------|
| <b>Chapter 1 INTRODUCTION .....</b>          | <b>1</b>  |
| <b>1.1 Project Overview.....</b>             | <b>1</b>  |
| <b>1.2 Objective .....</b>                   | <b>1</b>  |
| <b>1.3 Scope.....</b>                        | <b>2</b>  |
| <b>1.4 Technologies Used .....</b>           | <b>2</b>  |
| <b>Chapter 2 SYSTEM DESIGN.....</b>          | <b>3</b>  |
| <b>2.2 Schema Design.....</b>                | <b>4</b>  |
| <b>2.3 Normalization .....</b>               | <b>5</b>  |
| <b>Chapter 3 USER INTERFACE DESIGN .....</b> | <b>6</b>  |
| <b>Chapter 4 IMPLEMENTATION.....</b>         | <b>13</b> |
| <b>4.1 Database Creation.....</b>            | <b>13</b> |
| <b>4.1.1 MongoDB Scripts .....</b>           | <b>13</b> |
| <b>4.1.2 Table Structures .....</b>          | <b>13</b> |
| <b>4.2 Data Insertion.....</b>               | <b>15</b> |
| <b>4.3 Application Development .....</b>     | <b>20</b> |
| <b>4.3.1 Frontend.....</b>                   | <b>20</b> |
| <b>4.3.2 Backend.....</b>                    | <b>32</b> |
| <b>Chapter 5 CONCLUSION .....</b>            | <b>59</b> |
| <b>5.1 Summary of Work Done.....</b>         | <b>59</b> |

**5.2 Challenges Faced..... 60**

**5.3 Lessons Learned..... 61**

**5.4 Future Enhancements..... 62**

## **Chapter 1**

### **INTRODUCTION**

#### **1.1 Project Overview**

Facebook provides a platform for its users to share their stories, voice their opinions, and connect with others as part of its dedication to creating a sense of belonging among its users. The company strives to foster online communities where individuals can unite over shared interests, beliefs, or causes. However, the platform's algorithm is shown to have algorithmic opacity and bias, creating filter bubbles and echo chambers, keeping people inside their filter bubbles, deepening social divisions, and degrading public discourse.

Thus, our proposal for an improved Facebook mobile application allows its users to have control of the platform's algorithm, specifically, the content they see. Posts are now categorized into different categories to provide content tailored according to user preferences, which they can manage and modify. Users can report content and entities that do not meet community standards to decrease the rate at which such content is posted in their newsfeeds.

#### **1.2 Objective**

The system primarily aims to improve user empowerment by allowing personalized content delivery. Specifically, the goals of this project are:

- To provide a user-controlled content management system
- To implement a category-based feed system

These objectives combine user control and platform-driven content filtering to create a safer, more personalized content experience.

### **1.3 Scope**

The system proposal covers the functionalities of how Facebook generates users with content coming from textual posts of different users, pages, and groups. Thus, advertisements from third-party bodies are not covered by the system. Furthermore, user interactions are restricted to report and like posts only.

### **1.4 Technologies Used**

Java Programming Language through Eclipse IDE was utilized to establish interactions between the Graphical User Interface and MongoDB Database. Utilization of Figma focused on the Wireframing and Mockups of the GUI, which are then converted into functional frames through JavaFX and SceneBuilder.

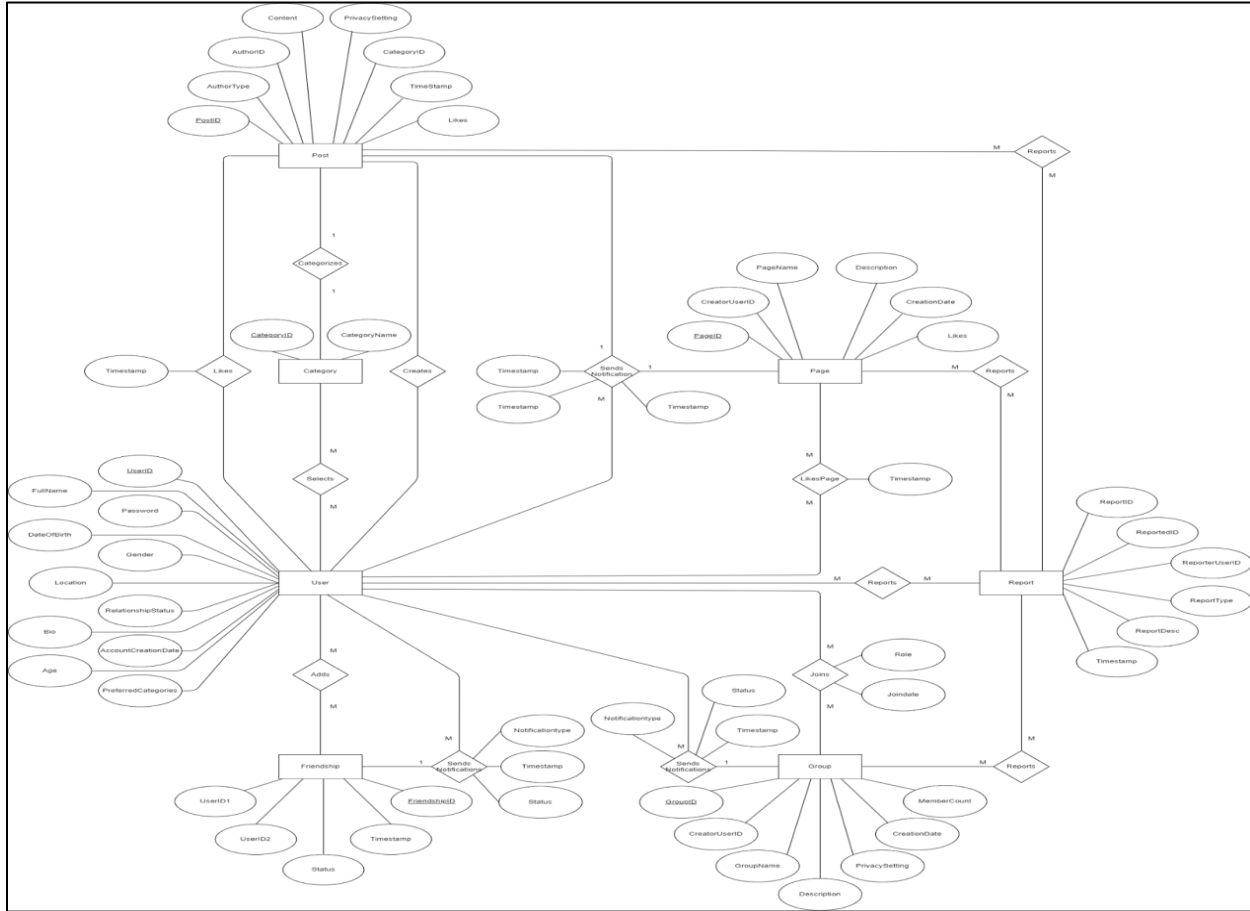
## Chapter 2

### SYSTEM DESIGN

#### 2.1 Entity-Relationship Diagram

**Figure 2.1**

*Entity-Relationship Diagram of Facebook Content Management System*

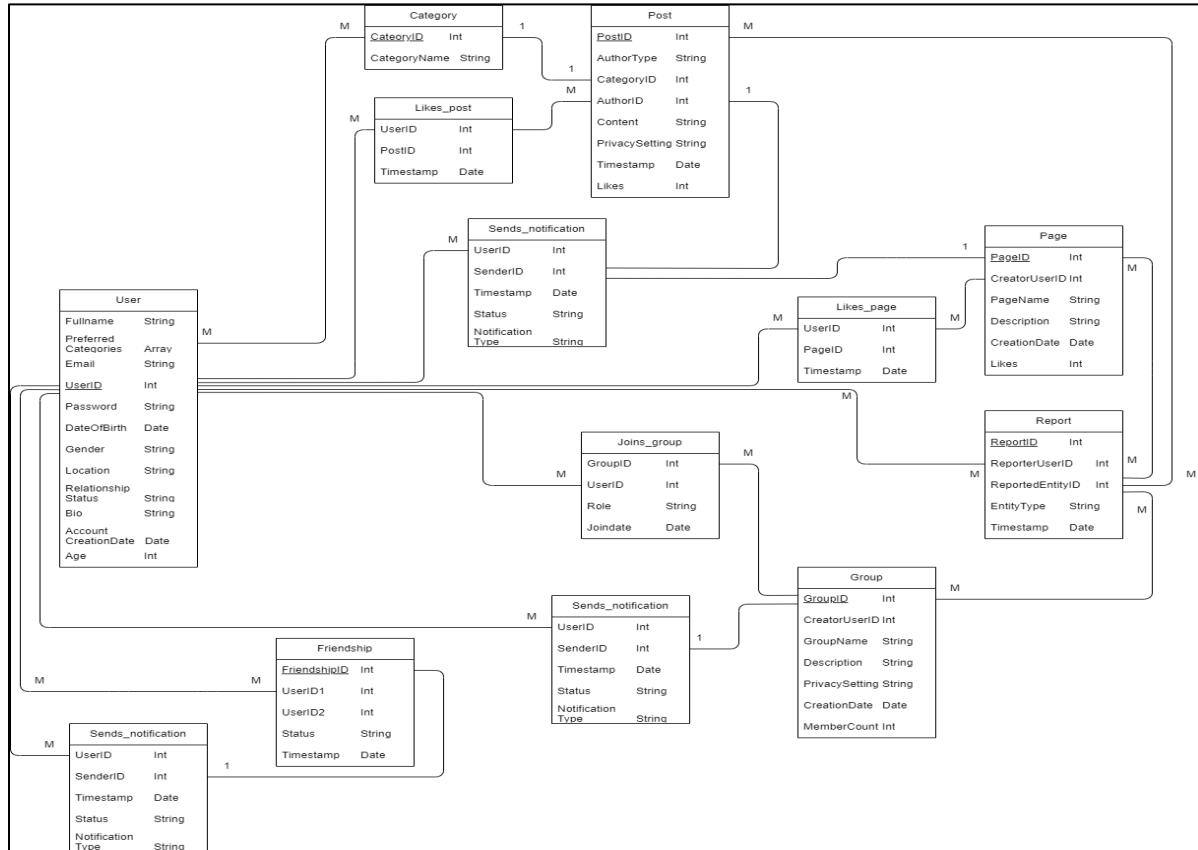


*Note:* The diagram consists of Seven Entities namely User, Report, Friendships, Category, Post, Page, and Group.

## 2.2 Schema Design

Figure 2.2

### *Schema Design of Facebook Content Management System*



## 2.3 Normalization

**Figure 2.3**

*Normalized Schema of Facebook Content Management System*

|              |              |                     |                |                  |               |               |                     |        |  |
|--------------|--------------|---------------------|----------------|------------------|---------------|---------------|---------------------|--------|--|
| User         |              |                     |                |                  |               |               |                     |        |  |
| UserID       | Username     | Email               | Password       | DateOfBirth      | Location      | Bio           | AccountCreationDate | Gender |  |
| Notification |              |                     |                |                  |               |               |                     |        |  |
| UserID       | SenderID     | NotificationType    | Timestamp      | Status           |               |               |                     |        |  |
| Friendship   |              |                     |                |                  |               |               |                     |        |  |
| FriendshipID | UserID1      | UserID2             | Status         | Timestamp        |               |               |                     |        |  |
| Category     |              |                     |                |                  |               |               |                     |        |  |
| CategoryID   | CategoryName | CategoryDescription |                |                  |               |               |                     |        |  |
| Post         |              |                     |                |                  |               |               |                     |        |  |
| PostID       | Content      | AuthorType          | Likes          | PrivacySetting   | Timestamp     | CategoryID    | AuthorID            |        |  |
| Report       |              |                     |                |                  |               |               |                     |        |  |
| ReportID     | EntityType   | Timestamp           | ReporterUserID | ReportedEntityID |               |               |                     |        |  |
| Group        |              |                     |                |                  |               |               |                     |        |  |
| GroupID      | GroupName    | Description         | PrivacySetting | MemberCount      | CreationDate  | CreatorUserID |                     |        |  |
| Page         |              |                     |                |                  |               |               |                     |        |  |
| PageID       | PageName     | Description         | CreationDate   | Likes            | CreatorUserID |               |                     |        |  |
| Joins_group  |              |                     |                |                  |               |               |                     |        |  |
| GroupID      | UserID       | Role                | JoinDate       |                  |               |               |                     |        |  |
| Likes_page   |              |                     |                |                  |               |               |                     |        |  |
| UserID       | PageID       | Timestamp           |                |                  |               |               |                     |        |  |
| Likes_post   |              |                     |                |                  |               |               |                     |        |  |
| UserID       | PostID       | Timestamp           |                |                  |               |               |                     |        |  |

## Chapter 3

### USER INTERFACE DESIGN

The following wireframes and mockups served as the reference for developing the User GUI through JavaFX and SceneBuilder.

**Figure 3.1**

*Login Page Wireframe and Mockup*

|                                                                                   |                                                                                    |
|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
|  |  |
| <input type="text" value="Mobile number or email"/>                               | <input type="text" value="Mobile number or email"/>                                |
| <input type="password" value="Password"/>                                         | <input type="password" value="Password"/>                                          |
| <input type="button" value="Login"/>                                              | <input type="button" value="Log in"/>                                              |
| <input type="button" value="Create new account"/>                                 | <input type="button" value="Create new account"/>                                  |

**Figure 3.2**

*Create Page Wireframe and Mockup*

|                                                                                     |                                                                                      |
|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
|  |  |
| <input type="text" value="Full Name (First name, Last name)"/>                      | <input type="text" value="Full Name (First name, Last name)"/>                       |
| <input type="text" value="MM/DD/YYYY"/>                                             | <input type="text" value="MM/DD/YYYY"/>                                              |
| <input type="text" value="Gender"/>                                                 | <input type="text" value="Gender"/>                                                  |
| <input type="text" value="Location"/>                                               | <input type="text" value="Location"/>                                                |
| <input type="text" value="Email"/>                                                  | <input type="text" value="Email"/>                                                   |
| <input type="password" value="Password"/>                                           | <input type="password" value="Password"/>                                            |
| <input type="password" value="Re-enter Password"/>                                  | <input type="password" value="Re-Enter Password"/>                                   |
| <input type="button" value="Sign Up"/>                                              | <input type="button" value="Sign Up"/>                                               |

Figure 3.3

Home Page Wireframe and Mockup

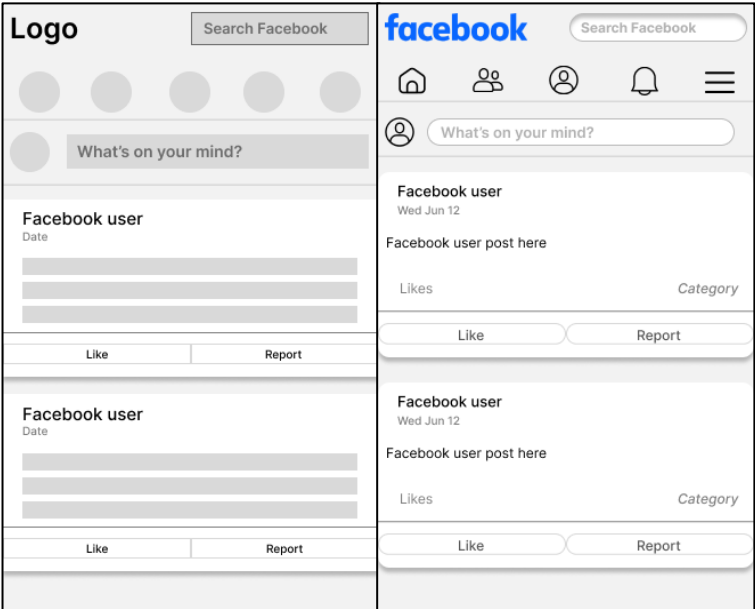


Figure 3.4

Friend List Wireframe and Mockup

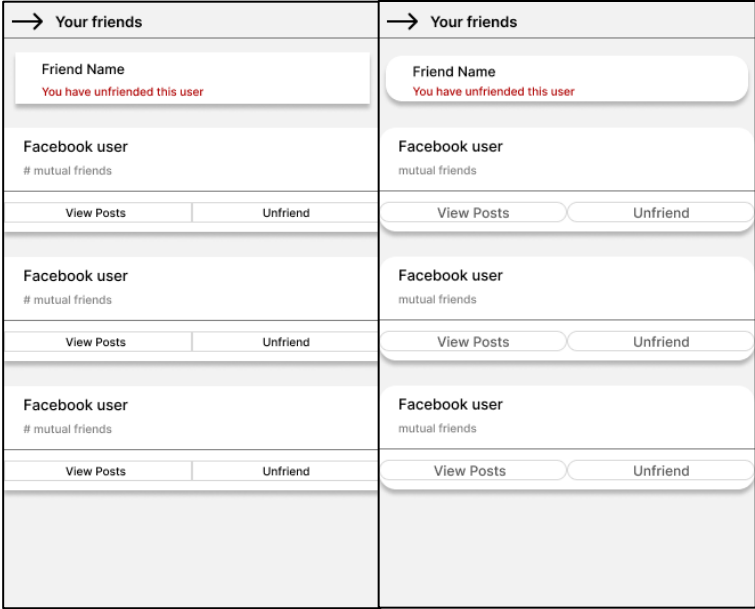


Figure 3.5

Friend Request Wireframe and Mockup

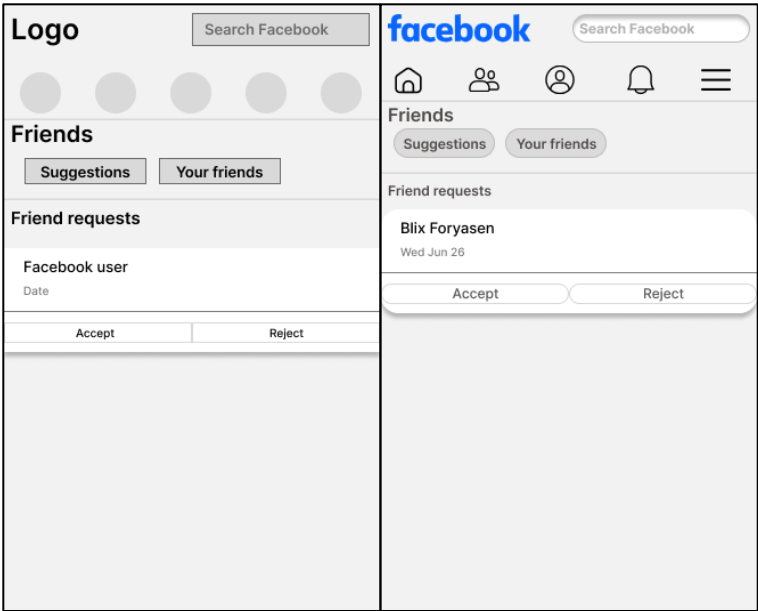
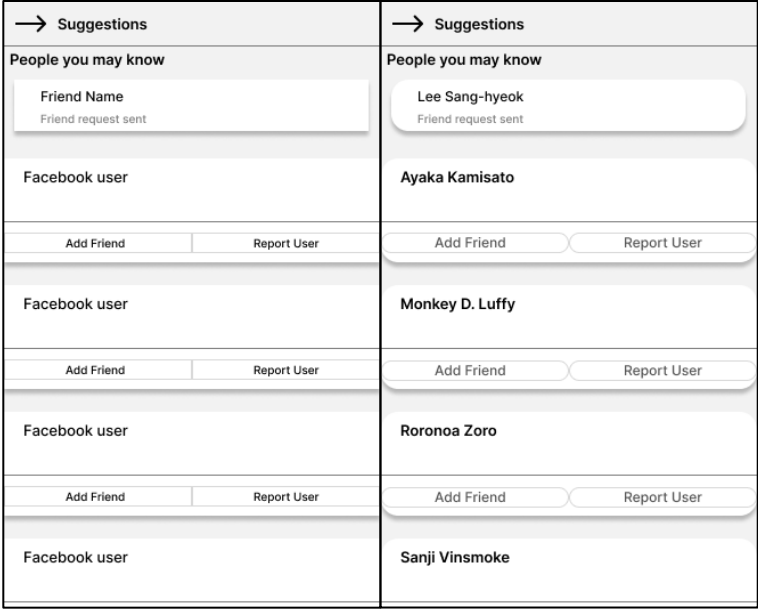


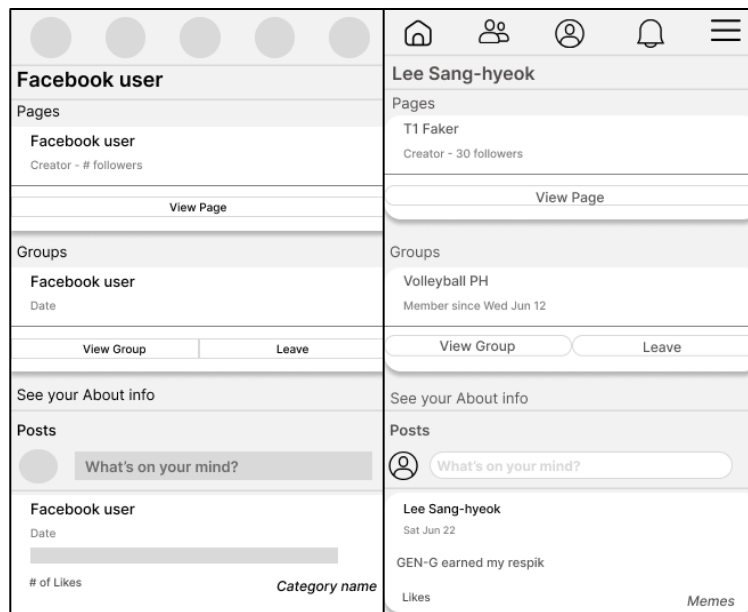
Figure 3.6

Suggestions Wireframe and Mockup



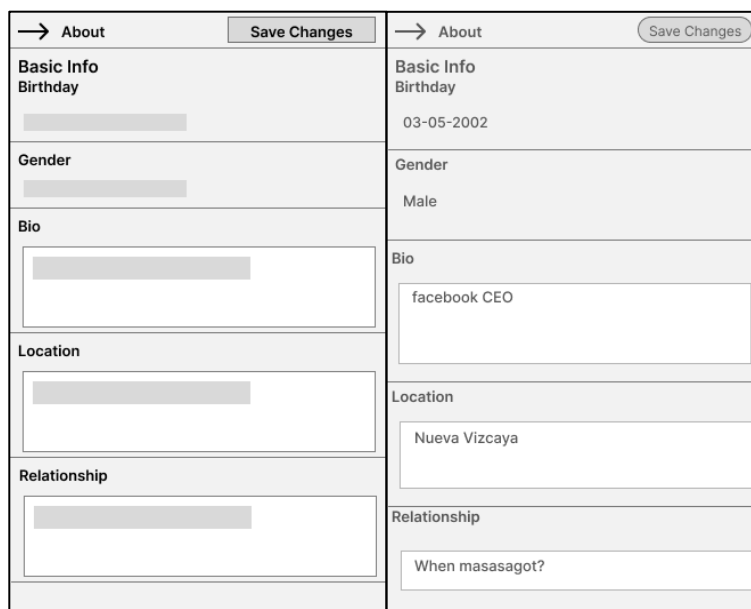
**Figure 3.7**

*User Profile Wireframe and Mockup*



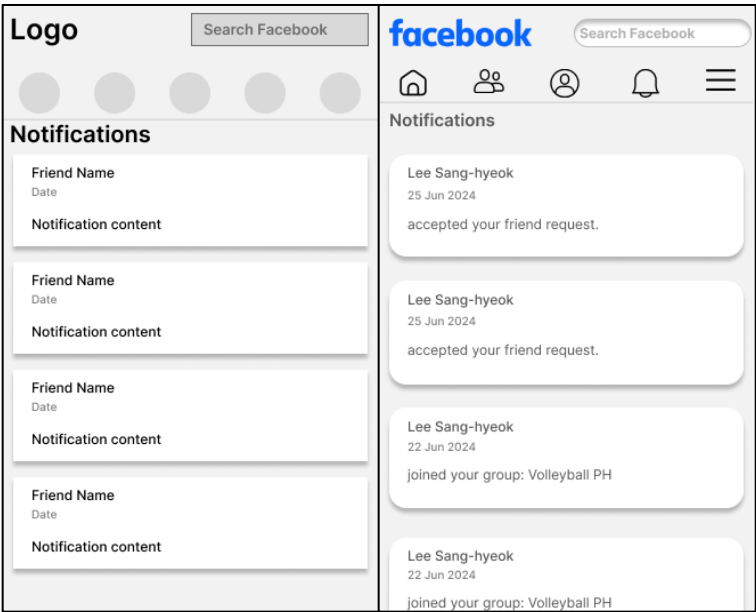
**Figure 3.8**

*About Info Wireframe and Mockup*



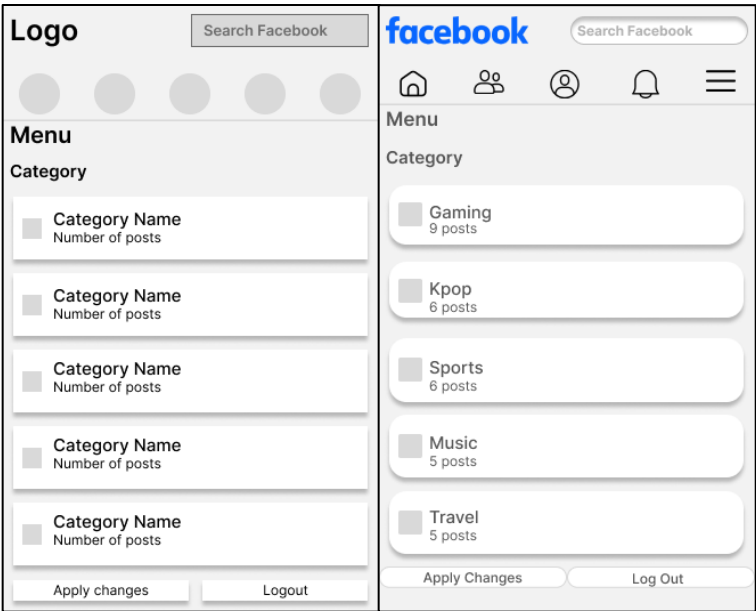
**Figure 3.9**

*Notification Wireframe and Mockup*



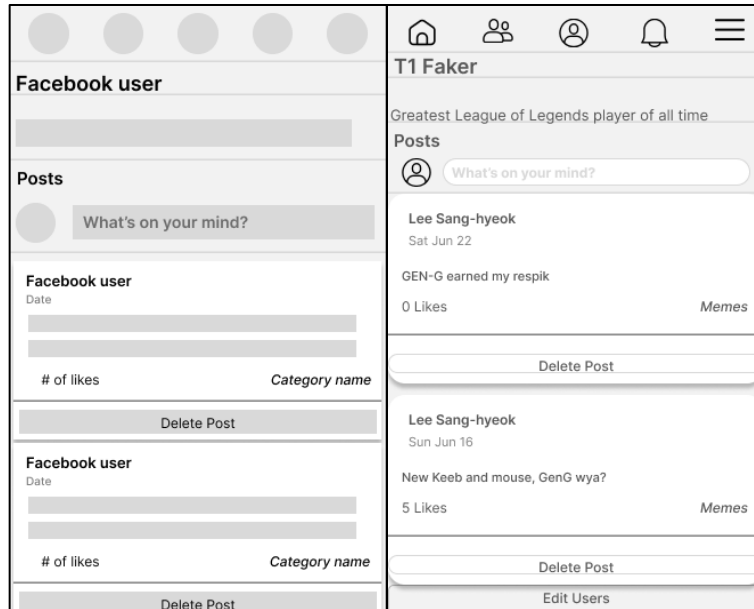
**Figure 3.10**

*Menu Wireframe and Mockup*



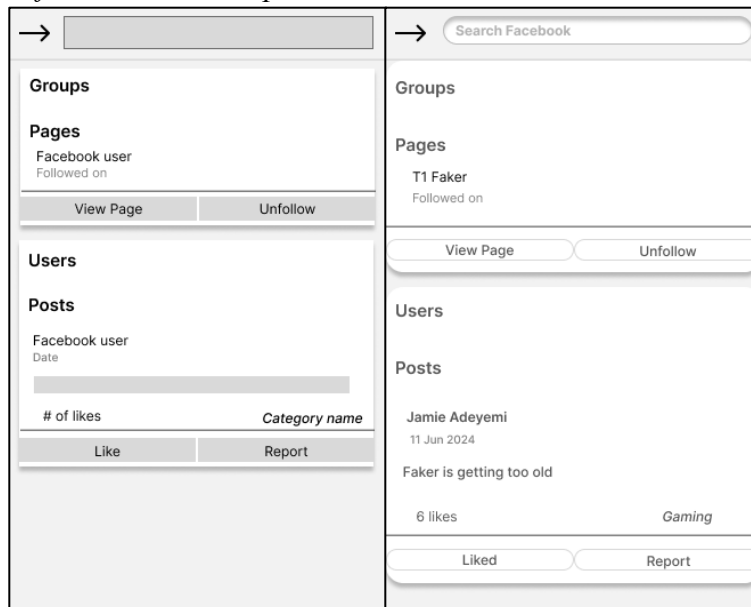
**Figure 3.11**

*Creator Wireframe and Mockup*



**Figure 3.12**

*Search Results Wireframe and Mockup*



**Figure 3.13**

*Search Results Wireframe and Mockup*

The image displays two side-by-side wireframe mockups of a 'Create post' form. Both mockups have a header bar with a back arrow, the text 'Create post', and a 'POST' button.

**Left Mockup (Generic):**

- Below the header, there is a user profile section with a circular icon placeholder, the text 'Name', and a 'Categories' dropdown menu.
- Below this is a large text input area with the placeholder text 'What's on your mind...'.
- The bottom half of the form is a large, empty rectangular area, likely for an image or video.

**Right Mockup (Facebook user):**

- The user profile section features a circular profile picture placeholder, the text 'Facebook user', and a 'Categories' dropdown menu.
- The text input area and the bottom image/video area are identical to the left mockup.

## Chapter 4

### IMPLEMENTATION

#### 4.1 Database Creation

##### 4.1.1 MongoDB Scripts

use FacebookContentManagement;

```
db.createCollection("user");

db.createCollection("page");

db.createCollection("pageLikes");

db.createCollection("post");

db.createCollection("category");

db.createCollection("group");

db.createCollection("groupMembers");

db.createCollection("notification");

db.createCollection("report");
```

##### 4.1.2 Table Structures

**Figure 4.1**

##### *User Collection*

|    |                                                         |          |
|----|---------------------------------------------------------|----------|
| 1  | _id: ObjectId('666869e28c5f544ecd3f17ce')               | ObjectId |
| 2  | UserID: 1                                               | Int32    |
| 3  | FullName: "Lee Sang-hyeok"                              | String   |
| 4  | Email: "faker@example.com"                              | String   |
| 5  | Password: "password123"                                 | String   |
| 6  | DateOfBirth: 2004-05-15T00:00:00.000+00:00              | Date     |
| 7  | Gender: "Male"                                          | String   |
| 8  | Location: "Sampaloc, Manila"                            | String   |
| 9  | RelationshipStatus: "Single"                            | String   |
| 10 | Bio: "I love league!"                                   | String   |
| 11 | AccountCreationTimestamp: 2024-06-11T15:14:42.462+00:00 | Date     |
| 12 | Age: 20                                                 | Int32    |
| 13 | ▼ PreferredCategories: Array (3)                        | Array    |
| 14 | 0: 1                                                    | Int32    |
| 15 | 1: 2                                                    | Int32    |
| 16 | 2: 3                                                    | Int32    |

**Figure 4.2**

*Category Collection*

|   |                                              |          |
|---|----------------------------------------------|----------|
| 1 | _id: ObjectId('666871ad8c5f544ecd3f17fe')    | ObjectId |
| 2 | CategoryID : 1                               | Int32    |
| 3 | CategoryName : "Technology"                  | String   |
| 4 | CreationDate : 2024-06-11T15:47:57.676+00:00 | Date     |

**Figure 4.3**

*Friendship Collection*

|   |                                           |          |
|---|-------------------------------------------|----------|
| 1 | _id: ObjectId('66688fc48c5f544ecd3f1900') | ObjectId |
| 2 | FriendshipID : 2                          | Int32    |
| 3 | UserID1 : 1                               | Int32    |
| 4 | UserID2 : 3                               | Int32    |
| 5 | Status : "Accepted"                       | String   |
| 6 | Timestamp : 2024-06-11T17:56:20.422+00:00 | Date     |

**Figure 4.4**

*Group Collection*

|   |                                              |          |
|---|----------------------------------------------|----------|
| 1 | _id: ObjectId('666878a88c5f544ecd3f17fe')    | ObjectId |
| 2 | GroupID : 1                                  | Int32    |
| 3 | CreatorUserID : 17                           | Int32    |
| 4 | GroupName : "Volleyball PH"                  | String   |
| 5 | Description : "Philippine Volleyball on top" | String   |
| 6 | PrivacySetting : "Public"                    | String   |
| 7 | CreationDate : 2024-06-11T16:17:44.449+00:00 | Date     |
| 8 | MemberCount : 30                             | Int32    |

**Figure 4.5**

*Group Members Collection*

|   |                                           |          |
|---|-------------------------------------------|----------|
| 1 | _id: ObjectId('666878a88c5f544ecd3f17ff') | ObjectId |
| 2 | GroupID : 1                               | Int32    |
| 3 | UserID : 17                               | Int32    |
| 4 | Role : "Creator"                          | String   |
| 5 | JoinDate : 2024-06-11T16:17:44.453+00:00  | Date     |

**Figure 4.6**

*Notification Collection*

|   |                                           |          |
|---|-------------------------------------------|----------|
| 1 | _id: ObjectId('6674498beb3102aaba3b9395') | ObjectId |
| 2 | UserID : 2                                | Int32    |
| 3 | SenderID : 1                              | Int32    |
| 4 | Timestamp : 2024-06-20T15:23:55.810+00:00 | Date     |
| 5 | Status : "Unread"                         | String   |
| 6 | NotificationType : "Friend request"       | String   |

**Figure 4.7**

*Page Collection*

|   |                                                               |          |
|---|---------------------------------------------------------------|----------|
| 1 | _id: ObjectId('66686f7b8c5f544ecd3f17ec')                     | ObjectId |
| 2 | PageID : 1                                                    | Int32    |
| 3 | CreatorUserID : 1                                             | Int32    |
| 4 | PageName : "T1 Faker"                                         | String   |
| 5 | Description : "Greatest League of Legends player of all time" | String   |
| 6 | CreationDate : 2024-06-11T15:38:35.421+00:00                  | Date     |
| 7 | Likes : 30                                                    | Int32    |

Figure 4.8

Page Likes Collection

|   |                                           |          |
|---|-------------------------------------------|----------|
| 1 | _id: ObjectId('66687b518c5f544ecd3f1812') | ObjectId |
| 2 | UserID : 1                                | Int32    |
| 3 | PageID : 1                                | Int32    |
| 4 | Timestamp : 2024-06-11T16:29:05.093+00:00 | Date     |

Figure 4.9

Post Collection

|   |                                           |          |
|---|-------------------------------------------|----------|
| 1 | _id: ObjectId('6668b2688c5f544ecd3f197d') | ObjectId |
| 2 | PostID : 1                                | Int32    |
| 3 | AuthorType : "User"                       | String   |
| 4 | AuthorID : 1                              | Int32    |
| 5 | Content : "Going back to 505"             | String   |
| 6 | PrivacySetting : "Public"                 | String   |
| 7 | CategoryID : 9                            | Int32    |
| 8 | Timestamp : 2024-06-11T20:24:08.877+00:00 | Date     |
| 9 | Likes : 5                                 | Int32    |

Figure 4.10

Post Likes Collection

|   |                                           |          |
|---|-------------------------------------------|----------|
| 1 | _id: ObjectId('666f0dfceb3102aaba3b91de') | ObjectId |
| 2 | UserID : 2                                | Int32    |
| 3 | PostID : 1                                | Int32    |
| 4 | Timestamp : 2024-06-16T16:08:28.458+00:00 | Date     |

Figure 4.11

Report Collection

|   |                                           |          |
|---|-------------------------------------------|----------|
| 1 | _id: ObjectId('66745fb06a614eae6eae6626') | ObjectId |
| 2 | ReportID : 1                              | Int32    |
| 3 | ReporterUserID : 1                        | Int32    |
| 4 | ReportedEntityID : 3                      | Int32    |
| 5 | EntityType : "User"                       | String   |
| 6 | Timestamp : 2024-06-20T16:58:24.317+00:00 | Date     |

## 4.2 Data Insertion

Figure 4.12

User Collection Sample Data

| _id | ObjectId                    | UserID | Int32 | FullName           | String | Email                  | String | Password         | String | DateOfBirth              | Date | Gender   | String |
|-----|-----------------------------|--------|-------|--------------------|--------|------------------------|--------|------------------|--------|--------------------------|------|----------|--------|
| 1   | ObjectId('666869e28c5f54... | 1      |       | "Lee Sang-hyeok"   |        | "faker@example.com"    |        | "password123"    |        | 2004-05-15T00:00:00.000+ |      | "Male"   |        |
| 2   | ObjectId('666869e28c5f54... | 2      |       | "Barbatos"         |        | "venti@example.com"    |        | "wind123"        |        | 2000-06-16T00:00:00.000+ |      | "Male"   |        |
| 3   | ObjectId('666869e28c5f54... | 3      |       | "Ayaka Kamisato"   |        | "ayaka@example.com"    |        | "ayaka123"       |        | 2001-09-28T00:00:00.000+ |      | "Female" |        |
| 4   | ObjectId('666869e28c5f54... | 4      |       | "Monkey D. Luffy"  |        | "luffy@example.com"    |        | "gumgum123"      |        | 1997-03-05T00:00:00.000+ |      | "Male"   |        |
| 5   | ObjectId('666869e28c5f54... | 5      |       | "Roronoa Zoro"     |        | "zoro@example.com"     |        | "threeblades123" |        | 1996-11-11T00:00:00.000+ |      | "Male"   |        |
| 6   | ObjectId('666869e28c5f54... | 6      |       | "Sanji Vinsmoke"   |        | "sanji@example.com"    |        | "blackleg123"    |        | 1997-03-02T00:00:00.000+ |      | "Male"   |        |
| 7   | ObjectId('666869e28c5f54... | 7      |       | "Brook Haven"      |        | "brook@example.com"    |        | "yohoho123"      |        | 1950-04-03T00:00:00.000+ |      | "Male"   |        |
| 8   | ObjectId('666869e28c5f54... | 8      |       | "Dracule Mihawk"   |        | "mihawk@example.com"   |        | "yoru123"        |        | 1980-03-09T00:00:00.000+ |      | "Male"   |        |
| 9   | ObjectId('666869e28c5f54... | 9      |       | "Silvers Rayleigh" |        | "rayleigh@example.com" |        | "haki123"        |        | 1938-05-13T00:00:00.000+ |      | "Male"   |        |
| 10  | ObjectId('666869e38c5f54... | 10     |       | "Kyllian Mbappe"   |        | "ky@example.com"       |        | "football123"    |        | 1998-12-20T00:00:00.000+ |      | "Male"   |        |

| Location String       | RelationshipStatus String | Bio String               | AccountCreationTimestamp ... | Age Int32 | PreferredCategories Array |
|-----------------------|---------------------------|--------------------------|------------------------------|-----------|---------------------------|
| "Sampaloc, Manila"    | "Single"                  | "I love league!"         | 2024-06-11T15:14:42.462+...  | 20        | [] 3 elements             |
| "Makati City"         | "Single"                  | "Bard of the wind."      | 2024-06-11T15:14:42.521+...  | 23        | [] 3 elements             |
| "Silang, Cavite"      | "Single"                  | "ayayayayaya"            | 2024-06-11T15:14:42.561+...  | 22        | [] 3 elements             |
| "Rio de Janeiro"      | "Single"                  | "mugimugi"               | 2024-06-11T15:14:42.616+...  | 27        | [] 3 elements             |
| "Kumamoto"            | "Single"                  | "yontoryuu"              | 2024-06-11T15:14:42.672+...  | 27        | [] 3 elements             |
| "Paris"               | "Single"                  | "number4"                | 2024-06-11T15:14:42.732+...  | 27        | [] 3 elements             |
| "Manchester"          | "Single"                  | "Yohohohoho"             | 2024-06-11T15:14:42.803+...  | 74        | [] 3 elements             |
| "Madrid"              | "Engaged"                 | "painter"                | 2024-06-11T15:14:42.882+...  | 44        | [] 3 elements             |
| "Sabaody Archipelago" | "Married"                 | "I h8 Marshall D. Teach" | 2024-06-11T15:14:42.952+...  | 86        | [] 3 elements             |
| "Paris"               | "Single"                  | "90second"               | 2024-06-11T15:14:43.016+...  | 25        | [] 3 elements             |

*Note:* A total of 30 documents are stored in the User Collection

**Figure 4.13**

### *Category Collection Sample Data*

| _id ObjectId                   | CategoryID Int32 | CategoryName String | CreationDate Date           |
|--------------------------------|------------------|---------------------|-----------------------------|
| 1 ObjectId('666871ad8c5f54...  | 1                | "Technology"        | 2024-06-11T15:47:57.676+... |
| 2 ObjectId('666871ad8c5f54...  | 2                | "Politics"          | 2024-06-11T15:47:57.710+... |
| 3 ObjectId('666871ad8c5f54...  | 3                | "Gaming"            | 2024-06-11T15:47:57.735+... |
| 4 ObjectId('666871ad8c5f54...  | 4                | "Cooking"           | 2024-06-11T15:47:57.764+... |
| 5 ObjectId('666871ad8c5f54...  | 5                | "Travel"            | 2024-06-11T15:47:57.793+... |
| 6 ObjectId('666871ad8c5f54...  | 6                | "Memes"             | 2024-06-11T15:47:57.815+... |
| 7 ObjectId('666871ad8c5f54...  | 7                | "Sports"            | 2024-06-11T15:47:57.838+... |
| 8 ObjectId('666871ad8c5f54...  | 8                | "Entertainment"     | 2024-06-11T15:47:57.863+... |
| 9 ObjectId('6668b2688c5f54...  | 9                | "Music"             | 2024-06-11T20:24:08.838+... |
| 10 ObjectId('6668b6318c5f54... | 10               | "Kpop"              | 2024-06-11T20:40:17.559+... |

*Note:* A total of 24 documents are stored in the Category Collection

**Figure 4.14**

### *Friendship Collection Sample Data*

| _id ObjectId                   | FriendshipID Int32 | UserID1 Int32 | UserID2 Int32 | Status String | Timestamp Date              |
|--------------------------------|--------------------|---------------|---------------|---------------|-----------------------------|
| 1 ObjectId('66688fc48c5f54...  | 2                  | 1             | 3             | "Accepted"    | 2024-06-11T17:56:20.422+... |
| 2 ObjectId('66688fc48c5f54...  | 3                  | 1             | 4             | "Accepted"    | 2024-06-11T17:56:20.514+... |
| 3 ObjectId('66688fc48c5f54...  | 4                  | 1             | 5             | "Accepted"    | 2024-06-11T17:56:20.611+... |
| 4 ObjectId('66688fc48c5f54...  | 5                  | 2             | 3             | "Accepted"    | 2024-06-11T17:56:20.700+... |
| 5 ObjectId('66688fc48c5f54...  | 6                  | 2             | 4             | "Accepted"    | 2024-06-11T17:56:20.787+... |
| 6 ObjectId('66688fc48c5f54...  | 7                  | 2             | 5             | "Accepted"    | 2024-06-11T17:56:20.867+... |
| 7 ObjectId('66688fc48c5f54...  | 8                  | 3             | 4             | "Accepted"    | 2024-06-11T17:56:20.957+... |
| 8 ObjectId('66688fc58c5f54...  | 9                  | 3             | 5             | "Accepted"    | 2024-06-11T17:56:21.049+... |
| 9 ObjectId('66688fc58c5f54...  | 10                 | 4             | 5             | "Accepted"    | 2024-06-11T17:56:21.145+... |
| 10 ObjectId('666891698c5f54... | 11                 | 6             | 7             | "Accepted"    | 2024-06-11T18:03:21.821+... |

*Note:* A total of 72 documents are stored in the Friendship Collection

**Figure 4.15**

*Group Collection Sample Data*

| _id | ObjectId                    | GroupID | Int32 | CreatorUserID | Int32 | GroupName                   | String | Description                 | String | PrivacySetting | String | CreationDate                | Date | MemberCount | Int32 |
|-----|-----------------------------|---------|-------|---------------|-------|-----------------------------|--------|-----------------------------|--------|----------------|--------|-----------------------------|------|-------------|-------|
| 1   | ObjectId('666878a88c5f54... | 1       |       | 17            |       | "Volleyball PH"             |        | "Philippine Volleyball o... |        | "Public"       |        | 2024-06-11T16:17:44.449+... |      | 30          |       |
| 2   | ObjectId('666878a88c5f54... | 2       |       | 21            |       | "Egypt Exploration"         |        | "bring a shield!"           |        | "Public"       |        | 2024-06-11T16:17:44.496+... |      | 4           |       |
| 3   | ObjectId('666878a88c5f54... | 3       |       | 22            |       | "Chemistry Group"           |        | "How to make the perfect... |        | "Public"       |        | 2024-06-11T16:17:44.534+... |      | 4           |       |
| 4   | ObjectId('666878a88c5f54... | 4       |       | 23            |       | "Revival"                   |        | "2nd Life"                  |        | "Public"       |        | 2024-06-11T16:17:44.552+... |      | 4           |       |
| 5   | ObjectId('666878a88c5f54... | 5       |       | 24            |       | "Valorant SK"               |        | "Valorant SK Community"     |        | "Public"       |        | 2024-06-11T16:17:44.578+... |      | 4           |       |
| 6   | ObjectId('666878a88c5f54... | 6       |       | 25            |       | "Forensics"                 |        | "Give me a corpse"          |        | "Public"       |        | 2024-06-11T16:17:44.613+... |      | 4           |       |
| 7   | ObjectId('666878a88c5f54... | 7       |       | 27            |       | "Genshin Impact Communit... |        | "Exploration, Quests, Ne... |        | "Public"       |        | 2024-06-11T16:17:44.635+... |      | 4           |       |
| 8   | ObjectId('666878a88c5f54... | 8       |       | 28            |       | "Chess"                     |        | "Chess Group"               |        | "Public"       |        | 2024-06-11T16:17:44.660+... |      | 4           |       |
| 9   | ObjectId('666878a88c5f54... | 9       |       | 29            |       | "Meta Design"               |        | "UI/UX group"               |        | "Public"       |        | 2024-06-11T16:17:44.697+... |      | 4           |       |
| 10  | ObjectId('666878a88c5f54... | 10      |       | 30            |       | "Bunnies"                   |        | "NewJeans fanbase"          |        | "Public"       |        | 2024-06-11T16:17:44.714+... |      | 4           |       |

*Note:* A total of 10 documents are stored in the Group Collection

**Figure 4.16**

*Group Members Collection Sample Data*

| _id | ObjectId                    | GroupID | Int32 | UserID | Int32 | Role      | String | JoinDate                    | Date |
|-----|-----------------------------|---------|-------|--------|-------|-----------|--------|-----------------------------|------|
| 1   | ObjectId('666878a88c5f54... | 1       |       | 17     |       | "Creator" |        | 2024-06-11T16:17:44.453+... |      |
| 2   | ObjectId('666878a88c5f54... | 2       |       | 21     |       | "Creator" |        | 2024-06-11T16:17:44.499+... |      |
| 3   | ObjectId('666878a88c5f54... | 3       |       | 22     |       | "Creator" |        | 2024-06-11T16:17:44.539+... |      |
| 4   | ObjectId('666878a88c5f54... | 4       |       | 23     |       | "Creator" |        | 2024-06-11T16:17:44.563+... |      |
| 5   | ObjectId('666878a88c5f54... | 5       |       | 24     |       | "Creator" |        | 2024-06-11T16:17:44.582+... |      |
| 6   | ObjectId('666878a88c5f54... | 6       |       | 25     |       | "Creator" |        | 2024-06-11T16:17:44.619+... |      |
| 7   | ObjectId('666878a88c5f54... | 7       |       | 27     |       | "Creator" |        | 2024-06-11T16:17:44.646+... |      |
| 8   | ObjectId('666878a88c5f54... | 8       |       | 28     |       | "Creator" |        | 2024-06-11T16:17:44.663+... |      |
| 9   | ObjectId('666878a88c5f54... | 9       |       | 29     |       | "Creator" |        | 2024-06-11T16:17:44.701+... |      |
| 10  | ObjectId('666878a88c5f54... | 10      |       | 30     |       | "Creator" |        | 2024-06-11T16:17:44.717+... |      |

*Note:* A total of 66 documents are stored in the Group Members Collection

**Figure 4.17**

*Notification Collection Sample Data*

| _id | ObjectId                    | UserID | Int32 | SenderID | Int32 | Timestamp                   | Date | Status   | String | NotificationType          | String |
|-----|-----------------------------|--------|-------|----------|-------|-----------------------------|------|----------|--------|---------------------------|--------|
| 1   | ObjectId('6674498beb3102... | 2      |       | 1        |       | 2024-06-20T15:23:55.810+... |      | "Unread" |        | "Friend request"          |        |
| 2   | ObjectId('667449b1eb3102... | 1      |       | 2        |       | 2024-06-20T15:24:33.390+... |      | "Unread" |        | "Friend request accepted" |        |
| 3   | ObjectId('66744a92eb3102... | 3      |       | 1        |       | 2024-06-20T15:28:18.677+... |      | "Unread" |        | "Friend request"          |        |
| 4   | ObjectId('66744a92eb3102... | 1      |       | 3        |       | 2024-06-20T15:28:18.690+... |      | "Unread" |        | "Friend request accepted" |        |
| 5   | ObjectId('66744a92eb3102... | 4      |       | 1        |       | 2024-06-20T15:28:18.760+... |      | "Unread" |        | "Friend request"          |        |
| 6   | ObjectId('66744a92eb3102... | 1      |       | 4        |       | 2024-06-20T15:28:18.783+... |      | "Unread" |        | "Friend request accepted" |        |
| 7   | ObjectId('66744a92eb3102... | 5      |       | 1        |       | 2024-06-20T15:28:18.843+... |      | "Unread" |        | "Friend request"          |        |
| 8   | ObjectId('66744a92eb3102... | 1      |       | 5        |       | 2024-06-20T15:28:18.856+... |      | "Unread" |        | "Friend request accepted" |        |
| 9   | ObjectId('66744a92eb3102... | 6      |       | 1        |       | 2024-06-20T15:28:18.926+... |      | "Unread" |        | "Friend request"          |        |
| 10  | ObjectId('66744a92eb3102... | 1      |       | 6        |       | 2024-06-20T15:28:18.950+... |      | "Unread" |        | "Friend request accepted" |        |

*Note:* A total of 692 documents are stored in the Notification Collection

**Figure 4.18**

*Page Collection Sample Data*

|    | _id ObjectId                | PageID Int32 | CreatorUserID Int32 | PageName String      | Description String          | CreationDate Date           | Likes Int32 |
|----|-----------------------------|--------------|---------------------|----------------------|-----------------------------|-----------------------------|-------------|
| 1  | ObjectId('66686f7b8c5f54... | 1            | 1                   | "T1 Faker"           | "Greatest League of Lege... | 2024-06-11T15:38:35.421+... | 30          |
| 2  | ObjectId('66686f7b8c5f54... | 2            | 2                   | "Anemo Bard"         | "Music concentrated page... | 2024-06-11T15:38:35.439+... | 3           |
| 3  | ObjectId('66686f7b8c5f54... | 3            | 3                   | "Handicrafts"        | "I make fans"               | 2024-06-11T15:38:35.450+... | 3           |
| 4  | ObjectId('66686f7b8c5f54... | 4            | 4                   | "Strawhat Pirates"   | "Kaizoku ou ni orewa nar... | 2024-06-11T15:38:35.474+... | 3           |
| 5  | ObjectId('66686f7b8c5f54... | 5            | 6                   | "Flambe Cuisine"     | "Cooking show"              | 2024-06-11T15:38:35.495+... | 3           |
| 6  | ObjectId('66686f7b8c5f54... | 6            | 7                   | "Soul King"          | "Heavy Metal"               | 2024-06-11T15:38:35.523+... | 3           |
| 7  | ObjectId('66686f7b8c5f54... | 7            | 8                   | "Painting by Mihawk" | "How to paint a sword"      | 2024-06-11T15:38:35.538+... | 3           |
| 8  | ObjectId('66686f7b8c5f54... | 8            | 9                   | "Ship Coating"       | "I just need the money t... | 2024-06-11T15:38:35.558+... | 3           |
| 9  | ObjectId('66686f7b8c5f54... | 9            | 10                  | "Real Madrid Mbatpe" | "2026WC is mine(real)"      | 2024-06-11T15:38:35.573+... | 3           |
| 10 | ObjectId('66686f7b8c5f54... | 10           | 30                  | "NewJeans"           | "How Sweet"                 | 2024-06-11T15:38:35.595+... | 3           |

*Note:* A total of 10 documents are stored in the Page Collection

**Figure 4.19**

*Page Likes Collection Sample Data*

|    | _id ObjectId                 | UserID Int32 | PageID Int32 | Timestamp Date               |
|----|------------------------------|--------------|--------------|------------------------------|
| 1  | ObjectId('66687b518c5f544... | 1            | 1            | 2024-06-11T16:29:05.093+0... |
| 2  | ObjectId('66687b518c5f544... | 2            | 1            | 2024-06-11T16:29:05.121+0... |
| 3  | ObjectId('66687b518c5f544... | 3            | 1            | 2024-06-11T16:29:05.143+0... |
| 4  | ObjectId('66687b518c5f544... | 4            | 2            | 2024-06-11T16:29:05.188+0... |
| 5  | ObjectId('66687b518c5f544... | 5            | 2            | 2024-06-11T16:29:05.213+0... |
| 6  | ObjectId('66687b518c5f544... | 6            | 2            | 2024-06-11T16:29:05.246+0... |
| 7  | ObjectId('66687b518c5f544... | 7            | 3            | 2024-06-11T16:29:05.276+0... |
| 8  | ObjectId('66687b518c5f544... | 8            | 3            | 2024-06-11T16:29:05.306+0... |
| 9  | ObjectId('66687b518c5f544... | 9            | 3            | 2024-06-11T16:29:05.331+0... |
| 10 | ObjectId('66687b518c5f544... | 10           | 4            | 2024-06-11T16:29:05.382+0... |

*Note:* A total of 57 documents are stored in the Page Likes Collection

**Figure 4.20**

*Post Collection Sample Data*

|    | _id ObjectId                | PostID Int32 | AuthorType String | AuthorID Int32 | Content String              | PrivacySetting String | CategoryID Int32 | Timestamp Date              |
|----|-----------------------------|--------------|-------------------|----------------|-----------------------------|-----------------------|------------------|-----------------------------|
| 1  | ObjectId('6668b2688c5f54... | 1            | "User"            | 1              | "Going back to 585"         | "Public"              | 9                | 2024-06-11T20:24:08.877+... |
| 2  | ObjectId('6668b6388c5f54... | 2            | "User"            | 2              | "helicopter helicopter"     | "Public"              | 2                | 2024-06-11T20:40:16.885+... |
| 3  | ObjectId('6668b6388c5f54... | 3            | "User"            | 3              | "too ez for PRX"            | "Public"              | 3                | 2024-06-11T20:40:16.950+... |
| 4  | ObjectId('6668b6318c5f54... | 4            | "User"            | 4              | "chicken oil"               | "Public"              | 4                | 2024-06-11T20:40:17.005+... |
| 5  | ObjectId('6668b6318c5f54... | 5            | "User"            | 5              | "flight soon!"              | "Public"              | 5                | 2024-06-11T20:40:17.054+... |
| 6  | ObjectId('6668b6318c5f54... | 6            | "User"            | 6              | "KEKW"                      | "Public"              | 6                | 2024-06-11T20:40:17.095+... |
| 7  | ObjectId('6668b6318c5f54... | 7            | "User"            | 7              | "MAVS in 6"                 | "Public"              | 7                | 2024-06-11T20:40:17.131+... |
| 8  | ObjectId('6668b6318c5f54... | 8            | "User"            | 8              | "why did the chicken cro... | "Public"              | 8                | 2024-06-11T20:40:17.185+... |
| 9  | ObjectId('6668b6318c5f54... | 9            | "User"            | 9              | "Sa'yo lang ang tingin"     | "Public"              | 9                | 2024-06-11T20:40:17.216+... |
| 10 | ObjectId('6668b6318c5f54... | 10           | "User"            | 10             | "Ipad AI calculator"        | "Public"              | 1                | 2024-06-11T20:40:17.269+... |

*Note:* A total of 70 documents are stored in the Post Collection

**Figure 4.21**

*Post Likes Collection Sample Data*

|    | _id ObjectId                | UserID Int32 | PostID Int32 | Timestamp Date              |
|----|-----------------------------|--------------|--------------|-----------------------------|
| 1  | ObjectId('666f0dfceb3102... | 2            | 1            | 2024-06-16T16:08:28.458+... |
| 2  | ObjectId('666f0dfceb3102... | 3            | 1            | 2024-06-16T16:08:28.508+... |
| 3  | ObjectId('666f0dfceb3102... | 4            | 1            | 2024-06-16T16:08:28.542+... |
| 4  | ObjectId('666f0dfceb3102... | 5            | 1            | 2024-06-16T16:08:28.562+... |
| 5  | ObjectId('666f0dfceb3102... | 6            | 1            | 2024-06-16T16:08:28.589+... |
| 6  | ObjectId('666f0dfceb3102... | 7            | 2            | 2024-06-16T16:08:28.620+... |
| 7  | ObjectId('666f0dfceb3102... | 8            | 2            | 2024-06-16T16:08:28.642+... |
| 8  | ObjectId('666f0dfceb3102... | 9            | 2            | 2024-06-16T16:08:28.674+... |
| 9  | ObjectId('666f0dfceb3102... | 10           | 2            | 2024-06-16T16:08:28.696+... |
| 10 | ObjectId('666f0dfceb3102... | 11           | 2            | 2024-06-16T16:08:28.727+... |

*Note:* A total of 216 documents are stored in the Post Likes Collection

**Figure 4.22**

*Report Collection Sample Data*

|    | _id ObjectId                | ReportID Int32 | ReporterUserID Int32 | ReportedEntityID Int32 | EntityType String | Timestamp Date              |
|----|-----------------------------|----------------|----------------------|------------------------|-------------------|-----------------------------|
| 1  | ObjectId('66745fb06a614e... | 1              | 1                    | 3                      | "User"            | 2024-06-20T16:58:24.317+... |
| 2  | ObjectId('66745fb06a614e... | 2              | 1                    | 3                      | "Page"            | 2024-06-20T16:58:24.337+... |
| 3  | ObjectId('66745fb06a614e... | 3              | 1                    | 2                      | "Post"            | 2024-06-20T16:58:24.358+... |
| 4  | ObjectId('66745fb06a614e... | 4              | 1                    | 5                      | "Group"           | 2024-06-20T16:58:24.376+... |
| 5  | ObjectId('66745fb06a614e... | 5              | 1                    | 5                      | "Post"            | 2024-06-20T16:58:24.393+... |
| 6  | ObjectId('66745fb06a614e... | 6              | 1                    | 6                      | "Post"            | 2024-06-20T16:58:24.410+... |
| 7  | ObjectId('66745fb06a614e... | 7              | 1                    | 17                     | "Post"            | 2024-06-20T16:58:24.430+... |
| 8  | ObjectId('66745fb06a614e... | 8              | 1                    | 8                      | "Group"           | 2024-06-20T16:58:24.446+... |
| 9  | ObjectId('66745fc46a614e... | 9              | 2                    | 29                     | "User"            | 2024-06-20T16:58:44.354+... |
| 10 | ObjectId('66745fc46a614e... | 10             | 2                    | 8                      | "Page"            | 2024-06-20T16:58:44.368+... |

*Note:* A total of 44 documents are stored in the Report Collection

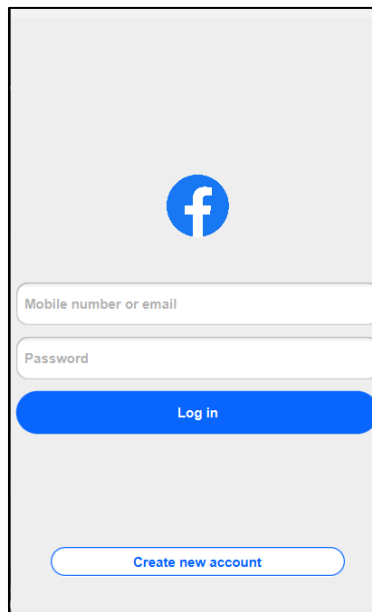
## 4.3 Application Development

### 4.3.1 Frontend

JavaFX accompanied by FXML through SceneBuilder served as the primary software for designing the front-end of the whole system. Each frame employed in the system consists of Java Swing and JavaFX components that shape the entire graphical user interface. Every post, notification, category, and connecting entity is designed separately and added dynamically into containers. VBox Layouts are implemented for each frame.

**Figure 4.23**

*Login Page Prototype Frame*



*Note:* User sees the login page frame as initialization for the frame

Frame Elements:

- VBox LoginPage
- ImageView fbLogo
- TextField email
- PasswordField password

- ToggleButton btnLogin
- btnCreateAccount

**Figure 4.24**

*Create Account Prototype Frame*

*Note:* The user creates an account by filling in necessary information

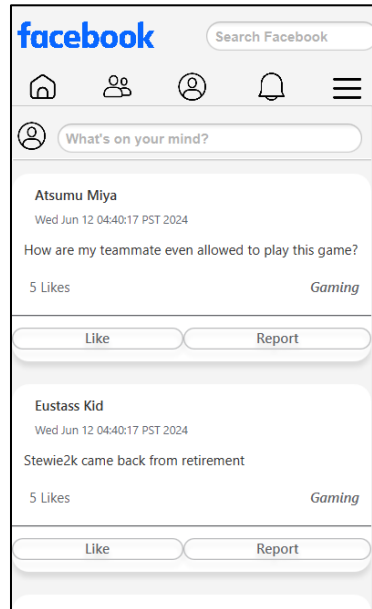
Frame Elements:

- VBox createAccountFrame
- ImageView fbLogo
- TextField txtFullName
- DatePicker calDate
- MenuButton cmbGender
- TextField txtLocation
- TextField txtEmail
- PasswordField pswrdField
- PasswordField reEnterPswrdField

- Button btnSignUp

**Figure 4.25**

*Home Page Prototype Frame*



*Note:* Upon Login, the user encounters the Home Page returning all relevant posts, whether category-based or the default feature, keeping up to date. Posts are dynamically added into the VBox postContainer.

Frame Elements for HomePage:

- VBox homePageFrame
- ImageView fbLogo
- ImageView home
- ImageView friends
- ImageView userProfile
- ImageView notification
- TextField searchField
- ImageView menu

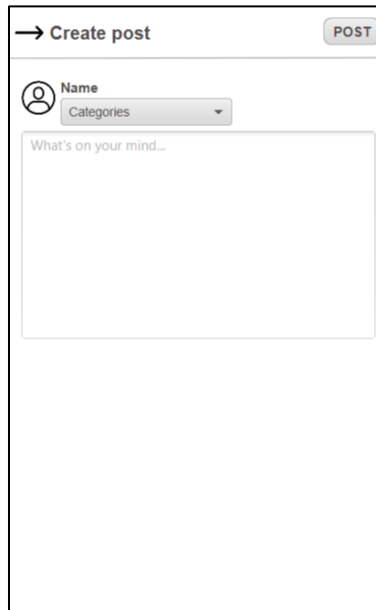
- ScrollPane
- VBox postContainer
- TextField btnPost
- ImageView friendProfile

Frame Elements for Post:

- VBox postVBox
- ImageView fbLogo
- Label username
- Label date
- Label caption
- StackPane reactionCounter
- Label nbReactions
- Label lblCategory
- HBox interactionBox
- HBox likeContainer
- Label likePrompt
- Hbox reportContainer
- Label reportPrompt

**Figure 4.26**

*Create Post Prototype Frame*



→ Create post POST

Name  
Categories

What's on your mind...

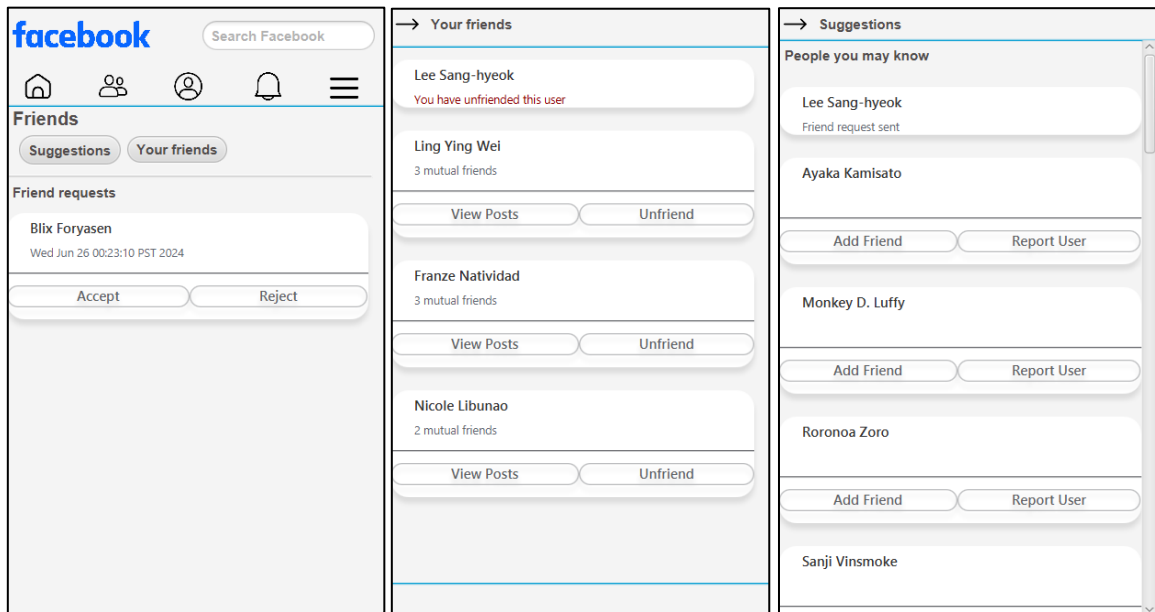
*Note:* Users create post by entering a post content and classifying it into a specific category

Frame Elements for CreatePostFrame:

- VBox postContainer
- ImageView btnBack
- Label lblCreatePost
- Button btnPost
- Label lblName
- MenuButton menuCategories
- TextArea txtPostContent

**Figure 4.27**

*Friend Request, Your Friend List, and Suggestions Prototype Frames*



*Note:* Users can view friend requests in the Friend Request Frame to accept or reject received friend requests. Moreover, they could view their friend list and suggested friends on the Your Friend List and Suggestions Frames, respectively. Entities are added dynamically into VBox containers per frame.

Frame Elements for FriendsFrame:

- VBox homePageFrame
- ImageView fbLogo
- ImageView home
- ImageView friends
- ImageView userProfile
- ImageView notification
- TextField searchField
- ImageView menu

- ScrollPane
- Label Friends
- Button suggestButton
- Button yourFriendsBtn
- Label FriendRequests
- VBox friendsContent

Frame Elements for EntitiesConnectedFrame (Your Friend List):

- VBox yourConnectionsContainer
- ImageView backButton
- Label lblConnection
- ScrollPane
- VBox yourConnectionsList

Frame Elements for SuggestionsFrame

- VBox suggestionsContainer
- ImageView backButton
- Label lblSuggestions
- ScrollPane
- VBox suggestedList
- Label peopleYouMayKnow

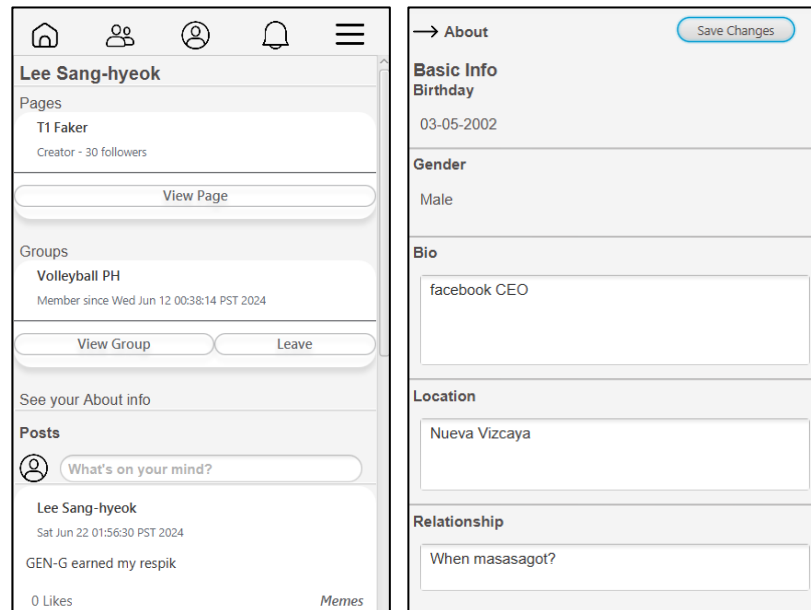
Frame Elements for ConnectEntities:

- VBox pendingEntityControl

- VBox vboxAccountDetails
- Label username
- Label date
- Label caption
- HBox reactionCounter
- HBox AcceptViewLikeJoinContainer
- Label AcceptViewLikeJoin
- HBox RejectUnfriendRemoveReportContainer
- Label RejectUnfriendRemoveReport

**Figure 4.28**

*User Profile Frame and Your About Info Prototype Frames*



*Note:* Users can view their profile in the User Profile Frame and edit or view their information in the Your About Info Frame. The User Profile Frame is also used to display Page and Group Information, with admin rights given to creators. Posts and Connected entities are dynamically added into respective VBox Containers.

Frame Elements for UserProfileFrame:

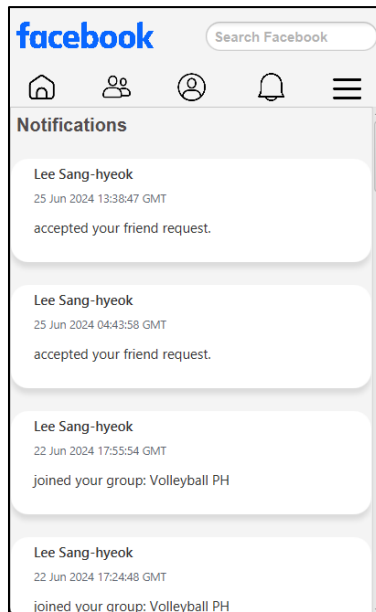
- VBox userProfileFrame
- ImageView home
- ImageView friends
- ImageView userProfile
- ImageView notification
- ImageView menu
- ScrollPane
- Label username
- VBox infoVBox
- Label lblPage
- VBox pagesLikedContainer
- Label lblGroups
- VBox groupsJoinedContainer
- Label aboutInfo
- VBox toPostDetails
- Label posts
- ImageView imgProfile
- TextField toPost
- VBox userPostsContainer
- Button btnEdit

Frame Elements for YourInfoFrame:

- VBox yourInfoContainer
- ImageView backButton
- Button btnSave
- Label lblBirthday
- Label lblGender
- TextArea txtBio
- TextArea txtLocation
- TextArea txtRelationship

**Figure 4.29**

*Notification Prototype Frame*



*Note:* Users view notifications and can mark if a certain notification has been read.

Notifications are added dynamically into VBox notificationContent.

Frame Elements for NotificationsFrame:

- VBox notificationsFrame

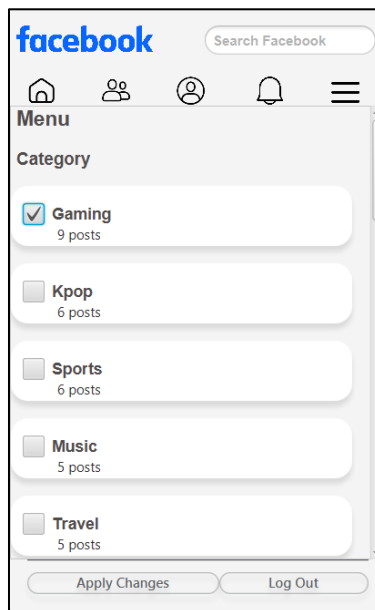
- ImageView fbLogo
- ImageView home
- ImageView friends
- ImageView userProfile
- ImageView notification
- TextField searchField
- ImageView menu
- ScrollPane
- VBox notificationContent

Frame Elements for Notification:

- VBox pendingEntityControl
- Label username
- Label date
- Label caption
- HBox btnAction

**Figure 4.30**

*Menu Prototype Frame*



*Note:* Users change preferred categories to enable the category-based feed or default news feed by the system. Categories shown are added dynamically given that posts exist under that category. Users may logout from this frame.

Frame Elements for MenuFrame:

- VBox menuFrame
- ImageView fbLogo
- ImageView home
- ImageView friends
- ImageView userProfile
- ImageView notification
- TextField searchField
- ImageView menu

- ScrollPane
- VBox categoryContainer
- HBox applyChangesContainer
- Label reactionName
- HBox btnLogout

Frame Elements for Category:

- CheckBox categoryEntity
- Label numberOfPosts

#### 4.3.2 Backend

MongoDB serve as the system's database for data retrieval, querying, updating, insertion, and removal. One entity corresponds to one collection in the database where data is stored in documents. Java Programming Language communicates to the Database through Maven dependencies. Each GUI has its controller to read user input and display information from the database.

**Figure 4.31**

*Maven Dependencies stored in pom.xml file for communication with MongoDB*

```
<dependencies>
  <dependency>
    <groupId>org.mongodb</groupId>
    <artifactId>mongodb-driver-sync</artifactId>
    <version>4.7.0</version>
  </dependency>
  <dependency>
    <groupId>ch.qos.logback</groupId>
    <artifactId>logback-classic</artifactId>
    <version>1.2.11</version>
  </dependency>
</dependencies>
```

**Figure 4.32**

### *Login Function*

```
@FXML
void btnLoginClicked(ActionEvent event) throws IOException {
    MongoClient mongoClient = MongoClient.create("mongodb://localhost:27017");
    MongoDBDatabase mongoDatabase = mongoClient.getDatabase("FacebookContentManagement");
    MongoCollection<Document> users = mongoDatabase.getCollection("users");

    Document foundDocument = users.find(Filters.eq("Email", email.getText())).first();

    if (email.getText().isBlank() || password.getText().isBlank()) {
        Alert alert = new Alert(Alert.AlertType.ERROR);
        alert.setTitle("Incomplete Details");
        alert.setHeaderText("Missing information detected");
        alert.setContentText("Please fill in the necessary information");
        alert.showAndWait();
    } else if (foundDocument != null && foundDocument.getString("Password").equals(password.getText())) {
        loggingUser.setUserDetails(foundDocument);
        Parent root = FXMLLoader.load(getClass().getResource("../FXML/HomePageFrame.fxml"));
        Stage stage = (Stage)((Node)event.getSource()).getScene().getWindow();
        Scene scene = new Scene(root);
        stage.setScene(scene);
        stage.show();
    } else {
        Alert alert = new Alert(Alert.AlertType.ERROR);
        alert.setTitle("Login Failed");
        alert.setHeaderText("Invalid Login Credentials");
        alert.setContentText("Please try again");
        alert.showAndWait();
    }

    mongoClient.close();
}
```

*Note:* Once the input fields are filled and the information the user provided is correct, the screen proceeds to the Home Page.

**Figure 4.33**

### *CurrentUser Class*

```
public class CurrentUser {
    private static final CurrentUser currentUser = new CurrentUser();
    private Document userDetails;
    private Document visitingProfile;
    private Document editingPost;
    private boolean isVisiting = false;
    private boolean isEditing = false;
    private List<Integer> likedPosts = new ArrayList<Integer>();
    private List<Integer> reportedPosts = new ArrayList<Integer>();
    private String searchedString;
```

*Note:* Upon login, the document matching the user's credentials is stored as reference for communication between the user and the database.

Figure 4.34

### Java Function to Create a New User

```
public static void createUser(String fullName, String email, String password, LocalDate dateOfBirth, String gender, String location, String relationshipStatus, String bio) {
    try (MongoClient mongoClient = MongoClient.create("mongodb://localhost:27017");) {
        MongoDB database = mongoClient.getDatabase("FacebookContentManagement");
        MongoCollection<Document> userCollection = database.getCollection("users");

        // Find the highest UserID
        Document highestUser = userCollection.find().sort(new Document("UserID", -1)).first();

        // Increment the UserID
        int newUserID = (highestUser != null) ? highestUser.getInteger("UserID") + 1 : 1;

        LocalDate currentDate = LocalDate.now();
        int age = Period.between(dateOfBirth, currentDate).getYears();

        // Create the user object
        Document user = new Document("UserID", newUserID)
            .append("FullName", fullName)
            .append("Email", email)
            .append("Password", password)
            .append("DateOfBirth", dateOfBirth)
            .append("Gender", gender)
            .append("Location", location)
            .append("RelationshipStatus", relationshipStatus)
            .append("Bio", bio)
            .append("AccountCreationTimestamp", new Date())
            .append("Age", age)
            .append("PreferredCategories", new ArrayList<Integer>());

        // Insert the user into the user collection
        InsertOneResult insertResult = userCollection.insertOne(user);

        if (insertResult.getInsertedId() == null) {
            throw new RuntimeException("Failed to insert user.");
        }

        System.out.println("User created successfully with UserID: " + newUserID);
    }
}
```

Note: Once all information required has been filled, a new user is inserted.

Figure 4.35

### Java Function for Retrieval of Posts from MongoDB

```
public static List<Document> getUserNewsfeed(Integer userId, List<Integer> selectedCategories) {
    // Create a MongoClient
    MongoClient mongoClient = MongoClient.create("mongodb://localhost:27017");

    // Access the database and collections
    MongoDB database = mongoClient.getDatabase("FacebookContentManagement");
    MongoCollection<Document> usersCollection = database.getCollection("users");
    MongoCollection<Document> friendshipsCollection = database.getCollection("friendship");
    MongoCollection<Document> followedPagesCollection = database.getCollection("pageLikes");
    MongoCollection<Document> joinedGroupsCollection = database.getCollection("groupMembers");
    MongoCollection<Document> categoriesCollection = database.getCollection("category");
    MongoCollection<Document> postsCollection = database.getCollection("post");
    MongoCollection<Document> groupCollection = database.getCollection("group");
    MongoCollection<Document> pageCollection = database.getCollection("page");
    MongoCollection<Document> postLikesCollection = database.getCollection("postLikes");
    MongoCollection<Document> reportCollection = database.getCollection("report");

    // Retrieve user's liked and reported posts
    FindIterable<Document> likedPosts = postLikesCollection.find(Filters.eq("UserID", userId));
    FindIterable<Document> reportedPosts = reportCollection.find(Filters.and(
        Filters.eq("ReporterUserID", userId), Filters.eq("EntityType", "Post")));
    FindIterable<Document> reportedUsers = reportCollection.find(Filters.and(
        Filters.eq("ReporterUserID", userId), Filters.eq("EntityType", "User")));
    FindIterable<Document> reportedPages = reportCollection.find(Filters.and(
        Filters.eq("ReporterUserID", userId), Filters.eq("EntityType", "Page")));
    FindIterable<Document> reportedGroups = reportCollection.find(Filters.and(
        Filters.eq("ReporterUserID", userId), Filters.eq("EntityType", "Group")));

    List<Integer> likedPostIDs = new ArrayList<>();
    for (Document likedPost : likedPosts) {
        likedPostIDs.add(likedPost.getInteger("PostID"));
    }

    loggedUser.setLikedPosts(likedPostIDs);

    List<Integer> reportedPostIDs = new ArrayList<Integer>();
    for (Document reportedPost : reportedPosts) {

```

```

        if(!reportedPostIDs.contains(reportedPost.getInteger("ReportedEntityID")))
            reportedPostIDs.add(reportedPost.getInteger("ReportedEntityID"));
    }
    loggedUser.setReportedPosts(reportedPostIDs);

    List<Integer> reportedUserIDs = new ArrayList<Integer>();
    for (Document reportedUser : reportedUsers) {
        if(!reportedUserIDs.contains(reportedUser.getInteger("ReportedEntityID")))
            reportedUserIDs.add(reportedUser.getInteger("ReportedEntityID"));
    }
    List<Integer> reportedPageIDs = new ArrayList<Integer>();
    for (Document reportedPage : reportedPages) {
        if(!reportedPageIDs.contains(reportedPage.getInteger("ReportedEntityID")))
            reportedPageIDs.add(reportedPage.getInteger("ReportedEntityID"));
    }
    List<Integer> reportedGroupIDs = new ArrayList<Integer>();
    for (Document reportedGroup : reportedGroups) {
        if(!reportedGroupIDs.contains(reportedGroup.getInteger("ReportedEntityID")))
            reportedGroupIDs.add(reportedGroup.getInteger("ReportedEntityID"));
    }

    // Retrieve the user's friends
    FindIterable<Document> friendships = friendshipsCollection.find(
        new Document("Status", "Accepted").append("$or", List.of(
            new Document("UserID1", userId),
            new Document("UserID2", userId)
        ))
    );

    List<Integer> friends = new ArrayList<>();
    for (Document friendship : friendships) {
        Integer userId1 = friendship.getInteger("UserID1");
        Integer userId2 = friendship.getInteger("UserID2");

        if (!userId1.equals(userId)) {
            friends.add(userId1);
        } else if (!userId2.equals(userId)) {
            friends.add(userId2);
        }
    }

    // Retrieve the pages followed by the user
    FindIterable<Document> followedPages = followedPagesCollection.find(Filters.eq("UserID", userId));
    List<Integer> followedPageIDs = new ArrayList<>();
    for (Document followedPage : followedPages) {
        followedPageIDs.add(followedPage.getInteger("PageID"));
    }

    // Retrieve the groups the user is a member of
    FindIterable<Document> groups = joinedGroupsCollection.find(Filters.eq("UserID", userId));
    List<Integer> groupIds = new ArrayList<>();
    for (Document group : groups) {
        groupIds.add(group.getInteger("GroupID"));
    }

```

```

// Fetch posts
List<Document> categoryPosts = null;
List<Document> friendPosts = null;
List<Document> pagePosts = null;
List<Document> groupPosts = null;

if (selectedCategories != null && !selectedCategories.isEmpty()) {
    // Fetch posts from friends with category filtering
    categoryPosts = postsCollection.find(Filters.and(
        new Document("CategoryID", new Document("$in", selectedCategories)),
        Filters.or(new Document("AuthorID", new Document("$nin", reportedUserIDs)),
            new Document("PageID", new Document("$nin", reportedPageIDs)),
            new Document("GroupID", new Document("$nin", reportedGroupIDs)))
    ).into(new ArrayList<>());
} else {
    // Fetch posts from friends without category filtering
    friendPosts = postsCollection.find(
        new Document("AuthorID", new Document("$in", friends))
    ).into(new ArrayList<>());

    // Fetch posts from followed pages without category filtering
    pagePosts = postsCollection.find(
        new Document("PageID", new Document("$in", followedPageIds))
    ).into(new ArrayList<>());

    // Fetch posts from groups without category filtering
    groupPosts = postsCollection.find(
        new Document("GroupID", new Document("$in", groupIds))
    ).into(new ArrayList<>());
}

// Combine all posts
List<Document> allPosts = new ArrayList<>();
if (selectedCategories != null && !selectedCategories.isEmpty())
    allPosts.addAll(categoryPosts);
else {
    allPosts.addAll(friendPosts);
    allPosts.addAll(pagePosts);
    allPosts.addAll(groupPosts);
}

Set<ObjectId> seenIds = new HashSet<>();
List<Document> newPosts = new ArrayList<>();
for (Document post : allPosts) {
    if (loggedUser.getLikedPosts().contains(post.getInteger("PostID")) || post.getInteger("AuthorID") == userId
        || loggedUser.getReportedPosts().contains(post.getInteger("PostID"))) //remove posts that are liked or reported
        continue;
    ObjectId postId = post.getObjectId("_id");
    if (!seenIds.contains(postId)) {
        seenIds.add(postId);
        newPosts.add(post);
    }
}

// Retrieve user names and category names
for (Document post : newPosts) {
    // Fetch user name from users collection
    Integer authorId = post.getInteger("AuthorID");
    Document user = usersCollection.find(new Document("UserID", authorId)).first();
    Document group = groupCollection.find(new Document("GroupID", post.getInteger("GroupID"))).first();
    Document page = pageCollection.find(new Document("PageID", post.getInteger("PageID"))).first();
    if (group != null) {
        post.put("FullName", user.getString("FullName"));
        post.put("GroupName", group.getString("GroupName"));
    } else if (page != null) {
        post.put("PageName", page.getString("PageName"));
    }
}

```

```

    }else
        post.put("FullName", user.getString("FullName"));

    // Fetch category name from categories collection
    Integer categoryId = post.getInteger("CategoryID");
    Document category = categoriesCollection.find(new Document("CategoryID", categoryId)).first()
    if (category != null) {
        post.put("CategoryName", category.getString("CategoryName"));
    }
    System.out.println(post.toJson());
}

// Sort unique posts by number of likes (descending) and then by date (descending)
newPosts.sort((post1, post2) -> {
    int likes1 = post1.getInteger("Likes", 0);
    int likes2 = post2.getInteger("Likes", 0);
    int likesComparison = Integer.compare(likes2, likes1); // Descending order
    if (likesComparison != 0) {
        return likesComparison;
    }
    return post2.getDate("Timestamp").compareTo(post1.getDate("Timestamp")); // Descending order
});
// Return the sorted list of posts
return newPosts;
}

```

*Note:* The default newsfeed involves checking the user's friends, liked pages, and joined groups. For the category-based feed, all posts are checked and only those under the specified categories are returned. All posts are up to date, excluding those reported and liked by the user. Posts coming from reported pages, users, and group are excluded.

**Figure 4.36**

### *Create Post Objects*

```

public List<Post> getPosts() {
    List<Integer> selectedCategories = loggedUser.getUserDetails().getList("PreferredCategories", Integer.class);
    List<Document> newsfeed = getUserNewsfeed(loggedUser.getUserDetails().getInteger("UserID"), selectedCategories);
    List<Post> ls = new ArrayList<Post>();
    for(Document postContents : newsfeed) {

        Post post = new Post();
        Account account = new Account();
        post.setPostID(postContents.getInteger("PostID"));
        account.setAccountID(postContents.getInteger("AuthorID"));
        post.setDate(postContents.getDate("Timestamp").toString());
        if(postContents.getString("GroupName") != null) {
            account.setName(postContents.getString("GroupName"));
            post.setDate(postContents.getString("FullName") + " - " + postContents.getDate("Timestamp").toString());
        }
        else if(postContents.getString("PageName") != null)
            account.setName(postContents.getString("PageName"));
        else
            account.setName(postContents.getString("FullName"));
        post.setAccount(account);
        post.setCategory(postContents.getString("CategoryName"));
        post.setCaption(postContents.getString("Content"));
        post.setTotalReactions(postContents.getInteger("Likes"));
        ls.add(post);
    }

    return ls;
}

```

*Note:* Documents retrieved from the database are then converted into Post Objects for data reference

**Figure 4.37**

*Post Class*

```
public class Post {
    private Account account;
    private int postID;
    private boolean likedPost;
    private String date;
    private String category;
    private String caption;
    private int totalReactions;
```

**Figure 4.38**

*Account Class*

```
public class Account {
    private int accountID;
    private String name;
    private int numberOfMembers;
    private Document visitingAccountDetails;
    private Document postDetails;
```

**Figure 4.39**

*Displaying Posts in User News Feed*

```
List<Post> posts;

static CurrentUser loggedInUser = CurrentUser.getCurrentUser();

@Override
public void initialize(URL location, ResourceBundle resources) {
    // TODO Auto-generated method stub
    posts = new ArrayList<>(getPosts());
    displayPosts();
    searchField.setFocusTraversable(false);
    btnPost.setFocusTraversable(false);
}

private void displayPosts() {
    try {
        for(Post publishedPost : posts) {
            FXMLLoader fxmlLoader = new FXMLLoader();
            fxmlLoader.setLocation(getClass().getResource("../FXML/post.fxml"));
            VBox vBox = fxmlLoader.load();
            PostController postController = fxmlLoader.getController();
            postController.setData(publishedPost);
            postContainer.getChildren().add(vBox);
        }
    } catch (IOException e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    }
}
```

*Note:* Each post object is converted into a VBox to be inserted in a Frame dynamically

**Figure 4.40**

### *Modifying JavaFX elements in Post VBox*

```
public void setData(Post post){
    this.post = post;
    username.setText(post.getAccount().getName());

    date.setText(post.getDate());

    lblCategory.setText(post.getCategory());

    if(post.getCaption() != null && !post.getCaption().isEmpty()){
        caption.setText(post.getCaption());
    }else{
        caption.setManaged(false);
    }

    if(loggedUser.getLikedPosts().contains(post.getPostID())) {
        likePrompt.setText("Liked");
        likePrompt.setTextFill(Color.rgb(66, 103, 178));
    }

    if(post.getAccount().getAccountID() == loggedUser.getUserDetails().getInteger("UserID")) {
        likePrompt.setText("Edit Post");
        reportPrompt.setText("Delete Post");
    }

    if(loggedUser.isVisiting() == true && loggedUser.getUserDetails().getInteger("UserID") == loggedUser.getVisitingProfile().getInteger("CreatorUserID")) {
        interactionBox.getChildren().remove(likeContainer);
        reportPrompt.setText("Delete Post");
    }

    nbReactions.setText(String.valueOf(post.getTotalReactions()) + " Likes");
}
```

*Note:* Displayed functionalities (e.g. Like, Edit, Delete) is dependent upon the user type

**Figure 4.41**

### *Java Function to Like a Post*

```
public void likePost(Integer userID, Integer postID) {
    try (MongoClient mongoClient = MongoClient.create("mongodb://localhost:27017")) {
        MongoDBDatabase db = mongoClient.getDatabase("FacebookContentManagement");
        MongoCollection<Document> postCollection = db.getCollection("post");
        MongoCollection<Document> groupMembersCollection = db.getCollection("groupMembers");
        MongoCollection<Document> postLikesCollection = db.getCollection("postLikes");
        MongoCollection<Document> pageCollection = db.getCollection("page");
        MongoCollection<Document> notificationCollection = db.getCollection("notification");

        // Find the post document by its ID
        Document post = postCollection.find(Filters.eq("PostID", postID)).first();

        if (post == null) {
            System.out.println("Error: Post not found.");
            return;
        }

        // Check if the like already exists
        Document existingLike = postLikesCollection.find(new Document("UserID", userID).append("PostID", postID)).first();
        if (existingLike != null) {
            System.out.println("Error: User has already liked this post.");
            return;
        }

        // Insert the like record
        Document like = new Document("UserID", userID)
            .append("PostID", postID)
            .append("Timestamp", new Date());

        postLikesCollection.insertOne(like);

        // Increment the Likes count
        int updatedLikes = post.getInteger("Likes", 0) + 1;

        // Update the post document with the incremented Likes count
        postCollection.updateOne(Filters.eq("PostID", postID), Updates.set("Likes", updatedLikes));

        // Determine the recipient of the notification
        Integer notificationUserID = null;
        if ("Page".equals(post.getString("AuthorType"))) {
            Document page = pageCollection.find(Filters.eq("PageID", post.getInteger("PageID"))).first();
            if (page != null) {
                notificationUserID = page.getInteger("CreatorUserID");
            } else {

```

```

        System.out.println("Error: Page not found.");
        return;
    }
    else if ("User".equals(post.getString("AuthorType"))) {
        notificationUserID = post.getInteger("AuthorID");
    }
    else {
        System.out.println("Error: Invalid AuthorType.");
        return;
    }

    // Create a notification
    Document notification = new Document("UserID", notificationUserID)
        .append("SenderID", userID)
        .append("PostID", postID)
        .append("Timestamp", new Date())
        .append("Status", "Unread")
        .append("NotificationType", "Post Like");

    notificationCollection.insertOne(notification);

    FindIterable<Document> likedPosts = postLikesCollection.find(Filters.eq("UserID", loggedUser.getUserDetails().getInteger("UserID")));
    List<Integer> likedPostIDs = new ArrayList<>();
    for (Document likedPost : likedPosts) {
        likedPostIDs.add(likedPost.getInteger("PostID"));
    }

    loggedUser.setLikedPosts(likedPostIDs);

    System.out.println("Post liked successfully by UserID: " + userID);
} catch (Exception e) {
    System.out.println("Error liking post: " + e);
}
}

```

*Note:* A new post like document is inserted into the “postLikes” collection before incrementing the like count of the post and sending a notification to its author.

**Figure 4.42**

### *Java Function to Remove a Like from a Post*

```

public void deleteLike(int userID, int postID) {
    MongoClient mongoClient = null;
    try {
        mongoClient = MongoClient.create("mongodb://localhost:27017");
        MongoDB database = mongoClient.getDatabase("FacebookContentManagement");
        MongoCollection<Document> postLikesCollection = database.getCollection("postLikes");
        MongoCollection<Document> postCollection = database.getCollection("post");

        // Find the like record
        Document like = postLikesCollection.find(new Document("UserID", userID).append("PostID", postID)).first();

        if (like == null) {
            System.out.println("Error: Like not found.");
            return;
        }

        // Delete the like record
        DeleteResult deleteResult = postLikesCollection.deleteOne(new Document("UserID", userID).append("PostID", postID));

        if (deleteResult.getDeletedCount() == 0) {
            System.out.println("Error: Failed to delete like.");
            return;
        }

        // Find the post document by its ID
        Document post = postCollection.find(Filters.eq("PostID", postID)).first();

        if (post == null) {
            System.out.println("Error: Post not found.");
            return;
        }

        // Decrement the Likes count
        int updatedLikes = post.getInteger("Likes") - 1;
    }
}

```

```

// Update the post document with the decremented Likes count
UpdateResult updateResult = postCollection.updateOne(
    Filters.eq("PostID", postID),
    Updates.set("Likes", updatedLikes)
);

if (updateResult.getModifiedCount() == 0) {
    System.out.println("Error: Failed to update Likes count in the posts collection.");
    return;
}

FindIterable<Document> likedPosts = postLikesCollection.find(Filters.eq("UserID", loggedUser.getUserDetails().getInteger("UserID")));
List<Integer> likedPostIDs = new ArrayList<>();
for (Document likedPost : likedPosts) {
    likedPostIDs.add(likedPost.getInteger("PostID"));
}

loggedUser.setLikedPosts(likedPostIDs);
System.out.println("Like deleted successfully by UserID: " + userID);
} catch (Exception e) {
    System.out.println("Error deleting like: " + e);
} finally {
    if (mongoClient != null) {
        mongoClient.close();
    }
}
}

```

*Note:* The post like is deleted from the “postLikes” collection before decrementing the like count of the post.

**Figure 4.43**

### *Java Function to Report a Post*

```

public void submitReport(int reporterUserID, int reportedEntityID, String entityType) {
    MongoClient mongoClient = null;
    try {
        mongoClient = MongoClient.create("mongodb://localhost:27017");
        MongoDBDatabase db = mongoClient.getDatabase("FacebookContentManagement");

        // Generate a new ReportID
        MongoCollection<Document> reportCollection = db.getCollection("Report");
        Document highestReport = reportCollection.find().sort(Sorts.descending("ReportID")).first();
        int newReportID = (highestReport != null) ? highestReport.getInteger("ReportID") + 1 : 1;

        // Create the report object
        Document report = new Document("ReportID", newReportID)
            .append("ReporterUserID", reporterUserID)
            .append("ReportedEntityID", reportedEntityID)
            .append("EntityType", entityType)
            .append("Timestamp", new Date());

        // Insert the report into the database
        reportCollection.insertOne(report);

        System.out.println("Report submitted successfully with ReportID: " + newReportID);

        FindIterable<Document> reportedPosts = reportCollection.find(Filters.and(
            Filters.eq("ReporterUserID", loggedUser.getUserDetails().getInteger("UserID")), Filters.eq("EntityType", "Post")));
        List<Integer> reportPostIDs = new ArrayList<>();
        for (Document reportedPost : reportedPosts) {
            System.out.println(reportedPost.getInteger("ReportedEntityID"));
            reportPostIDs.add(reportedPost.getInteger("ReportedEntityID"));
        }
        loggedUser.setReportedPosts(reportPostIDs);
    } catch (Exception e) {
        System.out.println("Error submitting report: " + e.getMessage());
    }
}

```

*Note:* A report document is stored in the “report” collection with the user as the reporter and the report type as a reported post.

**Figure 4.44**

### *Java Function to Insert a User Post*

```
public void createUserPost(int userID, String content, String categoryName) {
    try {
        // Check if the category exists
        MongoCollection<Document> categoryCollection = db.getCollection("category");
        Document category = categoryCollection.find(filters.regex("CategoryName", Pattern.compile("^" + categoryName + "$", Pattern.CASE_INSENSITIVE))).first();

        // Find the highest PostID
        MongoCollection<Document> postCollection = db.getCollection("post");
        Document highestPost = postCollection.find().sort(Sorts.descending("PostID")).first();

        // Increment the PostID
        int newPostID = (highestPost != null) ? highestPost.getInteger("PostID") + 1 : 1;

        // Create the post object
        Document post = new Document("PostID", newPostID)
            .append("AuthorType", "User")
            .append("AuthorID", userID)
            .append("Content", content)
            .append("PrivacySetting", "Public")
            .append("CategoryID", category.getInteger("CategoryID"))
            .append("Timestamp", new Date())
            .append("Likes", 0); // Initial likes is 0

        if (loggedUser.isVisiting() == true) {
            switch (loggedUser.getVisitingProfile().getString("EntityType")) {
                case "Group":
                    post.append("GroupID", loggedUser.getVisitingProfile().getInteger("GroupID"));
                    break;
                case "Page":
                    post.replace("AuthorType", "Page");
                    post.append("PageID", loggedUser.getVisitingProfile().getInteger("PageID"));
                    break;
            }
        }

        // Insert the post into the posts collection
        postCollection.insertOne(post);

        System.out.println("Post created successfully with PostID: " + newPostID);
    } catch (Exception e) {
        System.out.println("Error creating post: " + e.getMessage());
    }
}
```

*Note:* A document stores the information inputted by the user before insertion into the “users” collection. If the post is entered through a page or group, the post is linked to a certain page or group by their respective ID.

**Figure 4.45**

### *Java Function to Update a User Post*

```
public void updatePost(String content, String categoryName) {
    try {
        // Check if the category exists
        MongoCollection<Document> categoryCollection = db.getCollection("category");
        Document category = categoryCollection.find(filters.regex("CategoryName", Pattern.compile("^" + categoryName + "$", Pattern.CASE_INSENSITIVE))).first();

        MongoCollection<Document> postCollection = db.getCollection("post");

        Document editChanges = new Document("$set", new Document("CategoryID", category.getInteger("CategoryID"))
            .append("Content", content));

        UpdateResult updateResult = postCollection.updateOne(
            Filters.eq("PostID", loggedUser.getEditingPost().getInteger("PostID")),
            editChanges);

        if (updateResult.getModifiedCount() == 1)
            System.out.println("PostID " + loggedUser.getEditingPost().getInteger("PostID") + " edited successfully");
    } catch (Exception e) {
        e.printStackTrace();
    } finally {
        if (mongoClient != null) {
            mongoClient.close();
        }
    }
}
```

*Note:* A post’s content or category is editable by the user.

**Figure 4.46**

*Java Function to Delete a Post*

```
public void deletePost(int postID) {
    MongoClient mongoClient = null;
    try {
        mongoClient = MongoClient.create("mongodb://localhost:27017");
        MongoDB database = mongoClient.getDatabase("FacebookContentManagement");
        MongoCollection<Document> postCollection = database.getCollection("post");
        MongoCollection<Document> postLikesCollection = database.getCollection("postLikes");

        // Delete the post record
        DeleteResult deleteRecord = postCollection.deleteOne(new Document("PostID", postID));

        // Delete the postLikes record
        DeleteResult deleteLikes = postLikesCollection.deleteOne(new Document("PostID", postID));

        if (deleteRecord.getDeletedCount() == 0 && deleteLikes.getDeletedCount() == 0) {
            System.out.println("Error: Failed to delete post.");
            return;
        }
    } catch (Exception e) {
        System.out.println("Error deleting post: " + e);
    } finally {
        if (mongoClient != null) {
            mongoClient.close();
        }
    }
}
```

**Figure 4.47**

*Java Function to Retrieve all Pending Friend Requests by the User*

```
public static List<Document> getPendingFriendRequests(int userId) {
    List<Document> pendingRequests = new ArrayList<>();

    // Create a new MongoDB client
    try (MongoClient mongoClient = MongoClient.create("mongodb://localhost:27017")) {
        // Access the database
        MongoDB database = mongoClient.getDatabase("FacebookContentManagement");
        // Access the friendship requests collection
        MongoCollection<Document> friendshipRequestsCollection = database.getCollection("friendship");
        // Access the users collection
        MongoCollection<Document> usersCollection = database.getCollection("users");

        // Query to find pending friend requests
        Document query = new Document("UserID2", userId).append("Status", "Pending");

        // Execute the query and retrieve the results
        try (MongoCursor<Document> cursor = friendshipRequestsCollection.find(query).iterator()) {
            while (cursor.hasNext()) {
                Document request = cursor.next();
                int requesterID = request.getInteger("UserID1");

                // Find the full name of the requester
                Document userQuery = new Document("UserID", requesterID);
                Document user = usersCollection.find(userQuery).first();
                if (user != null) {
                    String fullName = user.getString("FullName");
                    request.append("FullName", fullName);
                }

                pendingRequests.add(request);
            }
        }
    }

    return pendingRequests;
}
```

*Note:* “UserID1” signifies the sender of the friend request while “UserID2” represents the receiver of the request.

**Figure 4.48**

*Connect Entities Class*

```
public class ConnectEntities {  
    private Account account;  
    private String date;  
    private String typeOfConnection;
```

**Figure 4.49**

*Java Functions to Convert and Display Retrieved Friend Requests into Objects*

```
private void displayFriendRequests() {  
    try {  
        for (ConnectEntities pendingRequests : pendingFriends) {  
            FXMLLoader fxmlloader = new FXMLLoader();  
            fxmlloader.setLocation(getClass().getResource("../FXML/ConnectEntities.fxml"));  
            VBox vbox = fxmlloader.load();  
            ConnectEntitiesController friendRequestController = fxmlloader.getController();  
            friendRequestController.setData(pendingRequests);  
            friendsContent.getChildren().add(vbox);  
        }  
    } catch (IOException e) {  
        // TODO Auto-generated catch block  
        e.printStackTrace();  
    }  
}  
  
//Friend Objects Creator  
public List<ConnectEntities> getFriendRequests() {  
    List<Document> pendingRequestFeed = getPendingFriendRequests(loggedUser.getUserDetails().getInteger("UserID"));  
    List<ConnectEntities> ls = new ArrayList<ConnectEntities>();  
    for (Document friendRequest : pendingRequestFeed) {  
        ConnectEntities pendingRequest;  
        pendingRequest = new ConnectEntities();  
        Account account = new Account();  
        account.setAccountID(friendRequest.getInteger("UserID1"));  
        account.setName(friendRequest.getString("FullName"));  
        pendingRequest.setAccount(account);  
        pendingRequest.setDate(friendRequest.getDate("Timestamp").toString());  
        pendingRequest.setTypeOfConnection("Pending Friend");  
        ls.add(pendingRequest);  
    }  
  
    return ls;  
}
```

*Note:* Objects turned into VBox are displayed into respective containers

**Figure 4.50**

*Java Function to Accept Pending Friend Requests*

```
public void acceptFriendRequest(int userID1, int userID2) {
    MongoClient mongoClient = null;
    try {
        mongoClient = MongoClient.create("mongodb://localhost:27017");
        MongoDB database = mongoClient.getDatabase("FacebookContentManagement");
        MongoCollection<Document> friendshipCollection = database.getCollection("friendship");
        MongoCollection<Document> notificationCollection = database.getCollection("notification");

        // Update friendship document
        UpdateResult friendshipResult = friendshipCollection.updateOne(
            Filters.and(
                Filters.eq("UserID1", userID1),
                Filters.eq("UserID2", userID2),
                Filters.eq("Status", "Pending")
            ),
            Updates.set("Status", "Accepted")
        );

        // Update corresponding notification status to "read"
        UpdateResult notificationResult = notificationCollection.updateOne(
            Filters.and(
                Filters.eq("UserID", userID2),
                Filters.eq("SenderID", userID1),
                Filters.eq("Status", "unread")
            ),
            Updates.set("Status", "read")
        );

        if (notificationResult.getModifiedCount() > 0) {
            System.out.println("Friendship status and corresponding notification status updated successfully.");
            // Create notification document
            Document notification = new Document("UserID", userID1)
                .append("SenderID", userID2)
                .append("Timestamp", new Date())
                .append("Status", "unread")
                .append("NotificationType", "Friend request accepted");

            notificationCollection.insertOne(notification);
        } else {
            System.out.println("Corresponding notification not found or already read.");
        }
    } catch (Exception e) {
        System.err.println("Error updating friendship status: " + e);
    } finally {
        // Close MongoDB connection in finally block to ensure it's always closed
        if (mongoClient != null) {
            mongoClient.close();
        }
    }
}
```

*Note:* Friendship status is changed to accepted and the sender of the friend request receives a notification about the acceptance of the friend request.

**Figure 4.51**

*Java Function to Remove Friends and Reject Friend Requests*

```
public void unfriendOrDecline(int userID1, int userID2) {
    MongoClient mongoClient = null;
    try {
        mongoClient = MongoClient.create("mongodb://localhost:27017");
        // Connect to your database
        MongoDB database = mongoClient.getDatabase("FacebookContentManagement");
        // Get reference to your collection
        MongoCollection<Document> friendshipCollection = database.getCollection("friendship");
        // Delete the friendship document
        long deletedCount = friendshipCollection.deleteOne(
            Filters.or(
                Filters.and(
                    Filters.eq("UserID1", userID1),
                    Filters.eq("UserID2", userID2),
                    Filters.or(Filters.eq("Status", "Accepted"), Filters.eq("Status", "Pending"))
                ),
                Filters.and(
                    Filters.eq("UserID1", userID2),
                    Filters.eq("UserID2", userID1),
                    Filters.or(Filters.eq("Status", "Accepted"), Filters.eq("Status", "Pending"))
                )
            )
        ).getDeletedCount();

        if (deletedCount == 0) {
            System.out.println("Error: No friendship found to remove.");
            return;
        }

        System.out.println("Friendship removed successfully between UserID: " + userID1 + " and UserID: " + userID2);
    } catch (Exception e) {
        System.out.println("Error unfrnding: " + e.getMessage());
    } finally {
        // Close MongoDB connection in finally block to ensure it's always closed
        if (mongoClient != null) {
            mongoClient.close();
        }
    }
}
```

*Note:* Entire friendship document is deleted from the “Friendships” collection

**Figure 4.52**

*Java Function to Retrieve Suggested Friends*

```
public static List<Document> getNonFriendUsersSortedByMutualFriends(int userID) {
    MongoClient mongoClient = null;
    List<Document> result = new ArrayList<>();

    try {
        // Connecting to MongoDB
        mongoClient = MongoClient.create("mongodb://localhost:27017");
        MongoDB database = mongoClient.getDatabase("FacebookContentManagement");
        MongoCollection<Document> friendshipCollection = database.getCollection("friendship");
        MongoCollection<Document> userCollection = database.getCollection("users");
        MongoCollection<Document> reportCollection = database.getCollection("report");

        FindIterable<Document> reportedUsers = reportCollection.find(Filters.and(
            Filters.eq("ReporterUserID", userID), Filters.eq("EntityType", "User")));
        List<Integer> reportedUserIDs = new ArrayList<Integer>();
        for (Document reportedUser : reportedUsers) {
            if (!reportedUserIDs.contains(reportedUser.getInteger("ReportedEntityID")))
                reportedUserIDs.add(reportedUser.getInteger("ReportedEntityID"));
        }

        // Creating the filter for accepted friendships
        Bson userFriendshipFilter = Filters.and(
            Filters.eq("Status", "Accepted"),
            Filters.or(Filters.eq("UserID1", userID), Filters.eq("UserID2", userID))
        );

        // Fetching the accepted friendships of the user
        List<Document> userAcceptedFriendships = friendshipCollection.find(userFriendshipFilter).into(new ArrayList<>());
        Set<Integer> userFriendIDs = new HashSet<>();
        for (Document friendship : userAcceptedFriendships) {
            int userID1 = friendship.getInteger("UserID1");
            int userID2 = friendship.getInteger("UserID2");
            if (userID1 == userID) {
                userFriendIDs.add(userID2);
            } else {
            }
        }
    }
}
```

```

    } else {
        userFriendIds.add(userID1);
    }
}

// Fetching users to whom the user has sent and accepted friend requests
Bson sentFriendRequestsFilter = Filters.or(
    Filters.and(
        Filters.eq("Status", "Pending"),
        Filters.eq("UserID1", userID)
    ), Filters.and(
        Filters.eq("Status", "Pending"),
        Filters.eq("UserID2", userID));
List<Document> friendRequests = friendshipCollection.find(sentFriendRequestsFilter).into(new ArrayList<>());

Set<Integer> sentFriendRequestIds = new HashSet<>();
for (Document request : friendRequests) {
    sentFriendRequestIds.add(request.getInteger("UserID2"));
    sentFriendRequestIds.add(request.getInteger("UserID1"));
}

// Fetching all users
List<Document> allUsers = userCollection.find().into(new ArrayList<>());

for (Document user : allUsers) {
    int otherUserID = user.getInteger("UserID");

    // Skip the user itself, user's friends, and reported users
    if (otherUserID == userID || userFriendIds.contains(otherUserID) || sentFriendRequestIds.contains(otherUserID) ||
        reportedUserIds.contains(otherUserID)) {
        continue;
    }

    String userName = user.getString("FullName");

    // Fetching the accepted friendships of each non-friend user
    Bson nonFriendFriendshipFilter = Filters.and(
        Filters.eq("Status", "Accepted"),
        Filters.or(Filters.eq("UserID1", otherUserID), Filters.eq("UserID2", otherUserID))
    );
    List<Document> nonFriendAcceptedFriendships = friendshipCollection.find(nonFriendFriendshipFilter).into(new ArrayList<>());

    // Calculating mutual friends
    Set<Integer> nonFriendFriendIds = new HashSet<>();
    for (Document friendship : nonFriendAcceptedFriendships) {
        int userID1 = friendship.getInteger("UserID1");
        int userID2 = friendship.getInteger("UserID2");
        if (userID1 == otherUserID) {
            nonFriendFriendIds.add(userID2);
        } else {
            nonFriendFriendIds.add(userID1);
        }
    }

    // Find mutual friends
    Set<Integer> mutualFriends = new HashSet<>(userFriendIds);
    mutualFriends.retainAll(nonFriendFriendIds);

    // Creating result document for this non-friend user
    Document resultDoc = new Document("UserName", userName)
        .append("NumberOfMutualFriends", mutualFriends.size()).append("UserID", otherUserID);
    result.add(resultDoc);

    // Sorting the result based on the number of mutual friends in descending order
    result.sort((doc1, doc2) -> doc2.getInteger("NumberOfMutualFriends") - doc1.getInteger("NumberOfMutualFriends"));
} catch (Exception e) {
    System.err.println("Error retrieving non-friend users and mutual friends: " + e.getMessage());
} finally {
    if (mongoClient != null) {
        mongoClient.close();
    }
}

return result;

```

*Note:* The function retrieves the suggested users based on the number of mutual friends the user has. Reported users are exempted from the suggestions.

**Figure 4.53**

*Java Function to Send a Friend Request to a User*

```
public void sendFriendRequest(int userID1, int userID2) {
    // Create MongoClient inside the method
    try (MongoClient mongoClient = MongoClient.create("mongodb://localhost:27017")) {
        MongoDB database = mongoClient.getDatabase("FacebookContentManagement");

        MongoCollection<Document> friendshipCollection = database.getCollection("friendship");
        MongoCollection<Document> notificationCollection = database.getCollection("notification");

        // Get the highest FriendshipID
        Document highestFriendship = friendshipCollection.find()
            .sort(Sorts.descending("FriendshipID"))
            .limit(1)
            .first();

        int newFriendshipID = (highestFriendship != null) ? highestFriendship.getInteger("FriendshipID") + 1 : 1;

        // Create friendship document
        Document friendship = new Document("FriendshipID", newFriendshipID)
            .append("UserID1", userID1)
            .append("UserID2", userID2)
            .append("Status", "Pending")
            .append("Timestamp", new Date());

        try {
            // Insert the friendship document into the collection
            friendshipCollection.insertOne(friendship);

            // Create notification document
            Document notification = new Document("UserID", userID2)
                .append("SenderID", userID1)
                .append("Timestamp", new Date())
                .append("Status", "unread")
                .append("NotificationType", "Friend request");

            notificationCollection.insertOne(notification);
        }
    }
}
```

*Note:* A friendship document is inserted with the current user as “UserID1” and the requested user as “UserID2.” Moreover, a notification is sent to the requested user informing them of the pending friend request.

**Figure 4.54**

*Java Function to get Posts by the Current User and All Associated Pages and Groups*

```
public static List<Document> getUserPosts(Integer userId) {
    // Create a MongoClient
    MongoClient mongoClient = MongoClient.create("mongodb://localhost:27017");

    // Access the database and collections
    MongoDB database = mongoClient.getDatabase("FacebookContentManagement");
    MongoCollection<Document> usersCollection = database.getCollection("users");
    MongoCollection<Document> categoriesCollection = database.getCollection("category");
    MongoCollection<Document> postsCollection = database.getCollection("post");

    // Fetch posts by user
    FindIterable<Document> postsByUser = null;
    if (loggedUser.isVisiting() == false)
        postsByUser = postsCollection.find(new Document("AuthorID", userId));
    else if (loggedUser.getVisitingProfile().getString("EntityType").equals("Group"))
        postsByUser = postsCollection.find(new Document("GroupID", userId));
    else if (loggedUser.getVisitingProfile().getString("EntityType").equals("Page"))
        postsByUser = postsCollection.find(new Document("PageID", userId));
    else if (loggedUser.getVisitingProfile().getString("EntityType").equals("Friend"))
        postsByUser = postsCollection.find(new Document("AuthorID", userId));
    List<Document> userPosts = new ArrayList<Document>();
    for (Document postDetails : postsByUser) {
        postDetails.append("EntityType", "Post");
        userPosts.add(postDetails);
    }
    // Sort unique posts by date (descending)
    userPosts.sort(Comparator.comparing((Document post) -> post.getDate("Timestamp")).reversed());
}
```

```

    // Fetch category name from categories collection
    for (Document post : userPosts) {
        Integer authorId = post.getInteger("AuthorID");
        Document user = usersCollection.find(new Document("UserID", authorId)).first();
        post.put("FullName", user.getString("FullName"));

        Integer categoryId = post.getInteger("CategoryID");
        Document category = categoriesCollection.find(new Document("CategoryID", categoryId)).first();
        if (category != null) {
            post.append("CategoryName", category.getString("CategoryName"));
        }
    }

    // Return userPosts
    return userPosts;
}

```

*Note:* Posts retrieved from the database are arranged by date in descending order

**Figure 4.55**

### *Java Function to Retrieve Liked Pages and Joined Groups by User*

```

public static List<Document> getUserPagesAndGroups(Integer userId) {
    // Create a MongoClient
    MongoClient mongoClient = MongoClient.create("mongodb://localhost:27017");

    // Access the database and collections
    MongoDB database = mongoClient.getDatabase("FacebookContentManagement");
    MongoCollection<Document> pageLikesCollection = database.getCollection("pageLikes");
    MongoCollection<Document> groupMemberCollection = database.getCollection("groupMembers");
    MongoCollection<Document> groupCollection = database.getCollection("group");
    MongoCollection<Document> pageCollection = database.getCollection("page");

    // Retrieve the pages followed by the user
    FindIterable<Document> followedPageIds = pageLikesCollection.find(Filters.eq("UserID", userId));
    List<Document> followedPages = new ArrayList<>();
    for (Document followedPage : followedPageIds) {
        followedPages.add(new Document(pageCollection.find(Filters.eq("PageID", followedPage.getInteger("PageID"))).first())
            .append("EntityType", "Page").append("Timestamp", followedPage.getDate("Timestamp")));
    }

    // Retrieve the groups the user is a member of
    FindIterable<Document> joinedGroupIds = groupMemberCollection.find(Filters.eq("UserID", userId));
    List<Document> joinedGroups = new ArrayList<>();
    for (Document group : joinedGroupIds) {
        joinedGroups.add(new Document(groupCollection.find(Filters.eq("GroupID", group.getInteger("GroupID"))).first())
            .append("Role", group.getString("Role")).append("EntityType", "Group")
            .append("Timestamp", group.getDate("JoinDate")));
    }

    // Combine all entities
    List<Document> allEntities = new ArrayList<>();
    allEntities.addAll(followedPages);
    allEntities.addAll(joinedGroups);

    // Return allEntities
    return allEntities;
}

```

*Note:* Compiles both entities into one List of Document Type, classifying them into groups and pages.

**Figure 4.56**

*Java Functions to Convert and Display Retrieved Pages and Groups into Objects*

```
public List<ConnectEntities> getPagesAndGroups() {
    List<Document> pagesAndGroupFeed = getUserPagesAndGroups(loggedUser.getUserDetails().getInteger("UserID"));
    List<ConnectEntities> pagesAndGroupObjects = new ArrayList<ConnectEntities>();
    for(Document pagesAndGroupContent : pagesAndGroupFeed) {
        ConnectEntities pageOrGroup = new ConnectEntities();
        Account account = new Account();
        switch (pagesAndGroupContent.getString("EntityType")) {
            case "Page":
                account.setAccountID(pagesAndGroupContent.getInteger("PageID"));
                account.setName(pagesAndGroupContent.getString("PageName"));
                account.setNumberOfMembers(pagesAndGroupContent.getInteger("Likes"));
                if(pagesAndGroupContent.getInteger("CreatorUserID") == (loggedUser.getUserDetails().getInteger("UserID")))
                    pageOrGroup.setTypeOfConnection("PageCreator");
                else
                    pageOrGroup.setTypeOfConnection("LikedPage");
                break;

            case "Group":
                account.setAccountID(pagesAndGroupContent.getInteger("GroupID"));
                account.setName(pagesAndGroupContent.getString("GroupName"));
                account.setNumberOfMembers(pagesAndGroupContent.getInteger("MemberCount"));
                if(pagesAndGroupContent.getString("Role").equals("Creator"))
                    pageOrGroup.setTypeOfConnection("Group Creator");
                else
                    pageOrGroup.setTypeOfConnection("Group Member");
                break;
        }

        account.setVisitingAccountDetails(pagesAndGroupContent);
        pageOrGroup.setAccount(account);
        pageOrGroup.setDate(pagesAndGroupContent.getDate("Timestamp").toString());
        pagesAndGroupObjects.add(pageOrGroup);
    }

    return pagesAndGroupObjects;
}

private void displayPagesAndGroups() {
    try {
        for(ConnectEntities pageOrGroup : pagesAndGroups) {
            FXMLLoader fxmlLoader = new FXMLLoader();
            fxmlLoader.setLocation(getClass().getResource("../FXML/ConnectEntities.fxml"));
            VBox vbox = fxmlLoader.load();
            ConnectEntitiesController entityController = fxmlLoader.getController();
            entityController.setData(pageOrGroup);
            switch (pageOrGroup.getTypeOfConnection()) {
                case "LikedPage":
                case "PageCreator":
                    pagesLikedContainer.getChildren().add(vbox);
                    break;

                case "Group Member":
                case "Group Creator":
                    groupsJoinedContainer.getChildren().add(vbox);
                    break;
            }
        }
    } catch (IOException e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    }
}
```

*Note:* Both pages and groups are converted into VBox before being dynamically inserted into their respective containers.

**Figure 4.57**

*Java Function to Like a Page*

```
public void likePage(int userID, int pageID) {
    MongoClient mongoClient = null;
    mongoClient = MongoClient.create("mongodb://localhost:27017");
    MongoDB database = mongoClient.getDatabase("FacebookContentManagement");
    MongoCollection<Document> pageCollection = db.getCollection("page");
    MongoCollection<Document> pageLikesCollection = db.getCollection("pageLikes");
    MongoCollection<Document> notificationCollection = db.getCollection("notification");

    try {
        // Find the page document by its ID
        Document page = pageCollection.find(new Document("PageID", pageID)).first();

        if (page == null) {
            System.out.println("Error: Page not found.");
            return;
        }

        // Increment the Likes count
        int updatedLikes = page.getInteger("Likes") + 1;

        // Update the page document with the incremented Likes count
        pageCollection.updateOne(new Document("PageID", pageID), new Document("$set", new Document("Likes", updatedLikes)));

        // Insert the like record
        Document like = new Document("UserID", userID)
            .append("PageID", pageID)
            .append("Timestamp", new Date());
        pageLikesCollection.insertOne(like);

        // Create a notification
        Document notification = new Document("UserID", page.getInteger("CreatorUserID"))
            .append("SenderID", userID)
            .append("Status", "Unread")
            .append("NotificationType", "Page Like")
            .append("Timestamp", new Date());
        notificationCollection.insertOne(notification);
    }
}
```

*Note:* The function inserts the new record into the “page like” collection, and increments the total number of likes, before sending a notification to the creator.

**Figure 4.58**

*Java Function to Unfollow a Page or remove a User from the Page*

```
public void unfollowOrRemoveFollower(int userID, int pageID) {
    MongoClient mongoClient = null;
    try {
        // Create the MongoClient
        mongoClient = MongoClient.create("mongodb://localhost:27017");
        MongoDB database = mongoClient.getDatabase("FacebookContentManagement");

        MongoCollection<Document> pageLikesCollection = database.getCollection("pageLikes");
        MongoCollection<Document> pageCollection = database.getCollection("page");

        // Delete the membership record
        DeleteResult deleteResult = pageLikesCollection.deleteOne(Filters.and(
            Filters.eq("UserID", userID),
            Filters.eq("PageID", pageID)
        ));

        // Decrement the MemberCount
        if (deleteResult.getDeletedCount() == 1) {
            pageCollection.updateOne(
                Filters.eq("PageID", pageID),
                Updates.inc("Likes", -1)
            );
        }

        System.out.println("UserID " + userID + " left group with PageID: " + pageID);
    } catch (Exception e) {
        System.out.println("Error leaving group: " + e);
    } finally {
        if (mongoClient != null) {
            mongoClient.close();
        }
    }
}
```

*Note:* The function removes the record from the “pageLikes” collection and decrements the total number of likes.

**Figure 4.59**

*Java Function to Join a Group*

```
public void joinGroup(int userID, int groupID) {
    MongoClient mongoClient = null;
    mongoClient = MongoClient.create("mongodb://localhost:27017");
    MongoDB database = mongoClient.getDatabase("FacebookContentManagement");
    MongoCollection<Document> groupCollection = db.getCollection("group");
    MongoCollection<Document> groupMembersCollection = db.getCollection("groupMembers");
    MongoCollection<Document> notificationCollection = db.getCollection("notification");

    try {
        // Check if the group exists
        Document group = groupCollection.find(new Document("GroupID", groupID)).first();

        // Add the user as a member of the group with the role "Member"
        Document member = new Document("GroupID", groupID)
            .append("UserID", userID)
            .append("Role", "Member")
            .append("JoinDate", new Date());

        groupMembersCollection.insertOne(member);

        // Increment the MemberCount in the group document
        groupCollection.updateOne(new Document("GroupID", groupID),
            new Document("$inc", new Document("MemberCount", 1)));

        // Create a notification for the group creator
        Document notification = new Document("UserID", group.getInteger("CreatorUserID"))
            .append("SenderID", userID)
            .append("Timestamp", new Date())
            .append("Status", "Unread")
            .append("NotificationType", "Join Group");

        notificationCollection.insertOne(notification);

        System.out.println("UserID " + userID + " joined group with GroupID: " + groupID);
    }
}
```

*Note:* The function inserts the new record into the “groupMembers” collection, and increments the total number of members, before sending a notification to the creator.

**Figure 4.60**

*Java Function to Leave a Group or Remove a User from the Group*

```
public void leaveOrRemoveGroup(int userID, int groupID) {
    MongoClient mongoClient = null;
    try {
        // Create the MongoClient
        mongoClient = MongoClient.create("mongodb://localhost:27017");
        MongoDB database = mongoClient.getDatabase("FacebookContentManagement");

        MongoCollection<Document> groupMembersCollection = database.getCollection("groupMembers");
        MongoCollection<Document> groupCollection = database.getCollection("group");

        // Delete the membership record
        DeleteResult deleteResult = groupMembersCollection.deleteOne(
            Filters.and(
                Filters.eq("UserID", userID),
                Filters.eq("GroupID", groupID)
            )
        );

        // Decrement the MemberCount
        if (deleteResult.getDeletedCount() == 1) {
            groupCollection.updateOne(
                Filters.eq("GroupID", groupID),
                Updates.inc("MemberCount", -1)
            );
        }

        System.out.println("UserID " + userID + " left group with GroupID: " + groupID);
    } catch (Exception e) {
        System.out.println("Error leaving group: " + e);
    } finally {
        if (mongoClient != null) {
            mongoClient.close();
        }
    }
}
```

*Note:* The function removes the record from the “groupMembers” collection and decrements the total number of members.

**Figure 4.61**

*Java Function to Update a User's Personal Info*

```
public void updateInfo(String bio, String location, String relationshipStatus) {
    MongoClient mongoClient = null;
    try {
        mongoClient = MongoClient.create("mongodb://localhost:27017");
        MongoDBDatabase db = mongoClient.getDatabase("FacebookContentManagement");
        // Check if the user exists

        MongoCollection<Document> userCollection = db.getCollection("users");

        Document editChanges = new Document("$set", new Document("Bio", bio)
            .append("Location", location)
            .append("RelationshipStatus", relationshipStatus));

        UpdateResult updateResult = userCollection.updateOne(
            Filters.eq("UserID", loggedUser.getUserDetails().getInteger("UserID")
            ), editChanges);

        if(updateResult.getModifiedCount() == 1) {
            System.out.println("UserID " + loggedUser.getUserDetails().getInteger("UserID") + " edited successfully");
            loggedUser.setUserDetails(userCollection.find(Filters.eq("UserID", loggedUser.getUserDetails().getInteger("UserID"))).first());
        }
    } catch (Exception e) {
        e.printStackTrace();
    } finally {
        if (mongoClient != null) {
            mongoClient.close();
        }
    }
}
```

*Note:* A user may edit their bio, relationship status, and location only

**Figure 4.62**

*Java Function to Show All Notifications for the Current User.*

```
public static List<Document> showNotificationsForUser(Integer userID) {
    MongoClient mongoClient = null;
    List<Document> notificationList = new ArrayList<>();
    try {
        // Establishing the connection to the database
        mongoClient = MongoClient.create("mongodb://localhost:27017");
        MongoDBDatabase database = mongoClient.getDatabase("FacebookContentManagement");
        MongoCollection<Document> collection = database.getCollection("notification");

        // Creating the query and finding the notifications
        Document query = new Document("UserID", userID);
        FindIterable<Document> notifications = collection.find(query).sort(new Document("Timestamp", -1));

        for (Document notification : notifications) {
            String notificationType = notification.getString("NotificationType");
            String SenderName = getSenderName(database, notification.getInteger("SenderID"));
            String PageOrGroupName = "";

            switch (notificationType) {
                case "Page Like":
                case "Page Post":
                    PageOrGroupName = getPageName(database, notification.getInteger("SenderID"));
                    break;
                case "Join Group":
                case "Group Post":
                    PageOrGroupName = getGroupName(database, notification.getInteger("SenderID"));
                    break;
            }

            notification.append("SenderName", SenderName);
            notification.append("PageOrGroupName", PageOrGroupName);

            notificationList.add(notification);
        }
    }
}
```

*Note:* Notifications for the User are classified based on their notification types.

Figure 4.63

*Java Functions to Convert and Display Notifications into Objects*

```
public List<Notification> getNotificationList() {
    List<Document> notifications = showNotificationsForUser(loggedUser.getUserDetails().getInteger("UserID"));
    List<Notification> ls = new ArrayList<Notification>();
    for(Document notification : notifications) {
        Notification addNotification;
        addNotification = new Notification();
        Account account = new Account();
        account.setName(notification.getString("SenderName"));
        account.setPostDetails(notification);
        addNotification.setAccount(account);
        addNotification.setDate(notification.getDate("Timestamp").toString());
        switch(notification.getString("NotificationType")) {
            case "Page Like":
                addNotification.setCaption("liked your page: " + notification.getString("PageOrGroupName"));
                break;
            case "Post Like":
                addNotification.setCaption("liked your post.");
                break;
            case "Friend request":
                addNotification.setCaption("sent you a friend request.");
                break;
            case "Join Group":
                addNotification.setCaption("joined your group: " + notification.getString("PageOrGroupName"));
                break;
            case "Group Post":
                addNotification.setCaption("posted in your group: " + notification.getString("PageOrGroupName"));
                break;
            case "Page Post":
                addNotification.setCaption("posted in your page: " + notification.getString("PageOrGroupName"));
                break;
            case "Friend request accepted":
                addNotification.setCaption("accepted your friend request.");
                break;
        }

        if(notification.getString("Status").equals("Unread"))
            addNotification.setRead(false);
        else
            addNotification.setRead(true);

        ls.add(addNotification);
    }

    return ls;
}

private void displayNotifications() {
    try {
        for(Notification notification : notificationList) {
            FXMLLoader fxmLoader = new FXMLLoader();
            fxmLoader.setLocation(getClass().getResource("../FXML/notification.fxml"));
            VBox vbox = fxmLoader.load();
            notificationController notifController = fxmLoader.getController();
            notifController.setData(notification);
            notificationContent.getChildren().add(vbox);
        }
    } catch (IOException e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    }
}
```

*Note:* All notifications are converted into VBox before being dynamically inserted into the notification container.

**Figure 4.64**

*Notification Class*

```
public class Notification {  
    private Account account;  
    private String date;  
    private String caption;  
    private boolean isRead;  
}
```

**Figure 4.65**

*Java Function to Update Status of Notification*

```
@FXML  
void onMarkClicked(MouseEvent event) {  
    MongoClient mongoClient = null;  
    try {  
        mongoClient = MongoClient.create("mongodb://localhost:27017");  
        MongoDB database = mongoClient.getDatabase("FacebookContentManagement");  
        MongoCollection<Document> notificationCollection = database.getCollection("notification");  
  
        UpdateResult notificationResult = notificationCollection.updateOne(  
            Filters.and(  
                Filters.eq("UserID", notificationPrompt.getAccount().getPostDetails().getInteger("UserID")),  
                Filters.eq("SenderID", notificationPrompt.getAccount().getPostDetails().getInteger("SenderID")),  
                Filters.eq("Timestamp", notificationPrompt.getAccount().getPostDetails().getDate("Timestamp")),  
                Filters.eq("Status", notificationPrompt.getAccount().getPostDetails().getString("Status")),  
                Filters.eq("Status", "Unread")  
            ),  
            Updates.set("Status", "Read")  
        );  
  
        if (notificationResult.getModifiedCount() > 0) {  
            System.out.println("Notification Read");  
            promptRead.getChildren().remove(btnAction);  
        } else {  
            System.out.println("Corresponding notification not found or already read.");  
        }  
    } catch (Exception e) {  
        // TODO: handle exception  
        System.err.println("Error updating status: " + e);  
    }  
}
```

*Note:* Updates notification status of notification from “Unread” to “Read”

**Figure 4.66**

*Java Function to Get List of Members or Followers*

```
public static List<Document> getConnectionEntities(int userID) {
    MongoClient mongoClient = null;
    List<Document> result = new ArrayList<Document>();
    try {
        // Connecting to MongoDB
        mongoClient = MongoClient.create("mongodb://localhost:27017");
        MongoDB database = mongoClient.getDatabase("FacebookContentManagement");
        MongoCollection<Document> userCollection = database.getCollection("users");
        MongoCollection<Document> followedPagesCollection = database.getCollection("pageLikes");
        MongoCollection<Document> joinedGroupsCollection = database.getCollection("groupMembers");

        Bson followerFilter = Filters.eq("PageID", userID);
        Bson memberFilter = Filters.and(Filters.eq("GroupID", userID), Filters.eq("Role", "Member"));

        // Fetching the follower/memberIDs of the user
        FindIterable<Document> followedPages = followedPagesCollection.find(followerFilter);
        FindIterable<Document> joinedGroups = joinedGroupsCollection.find(memberFilter);

        // Fetching names of connections
        List<Document> followers = new ArrayList<Document>();
        List<Document> members = new ArrayList<Document>();

        for(Document followerID : followedPages) {
            Document followerProfile = userCollection.find(new Document("UserID", followerID.getInteger("UserID"))).first();
            followers.add(followerProfile);
        }

        for(Document memberID : joinedGroups) {
            Document memberProfile = userCollection.find(new Document("UserID", memberID.getInteger("UserID"))).first();

            members.add(memberProfile);
        }

        if(loggedUser.getVisitingProfile().getString("EntityType").equals("Group"))
            result = members;
        else if(loggedUser.getVisitingProfile().getString("EntityType").equals("Page"))
            result = followers;
    }
}
```

*Note:* The function returns all followers of Pages or Members of Groups for viewing of the Page or Group Creator

**Figure 4.67**

*Java Function to Retrieve Categories with Posts*

```
public static List<Document> countPostsPerCategory() {
    AggregateIterable<Document> postCounts = null;
    MongoClient mongoClient = null;
    List<Document> categoryList = new ArrayList<>();
    try {
        mongoClient = MongoClient.create("mongodb://localhost:27017");
        MongoDB database = mongoClient.getDatabase("FacebookContentManagement");
        MongoCollection<Document> postsCollection = database.getCollection("post");

        postCounts = postsCollection.aggregate(Arrays.asList(
            Aggregates.group("$CategoryID", Accumulators.sum("count", 1)),
            Aggregates.lookup("category", "id", "CategoryID", "category"),
            Aggregates.project(Projections.fields(
                Projections.include("CategoryID"),
                Projections.computed("CategoryName", new Document("$arrayElemAt", Arrays.asList("$category.CategoryName", 0))),
                Projections.computed("postCount", "$count")
            )),
            Aggregates.sort(Sorts.descending("postCount"))
        ));

        for(Document category : postCounts) {
            categoryList.add(category);
        }
    } catch (Exception e) {
        System.out.println("Error counting posts per category: " + e.getMessage());
    } finally {
        if (mongoClient != null) {
            mongoClient.close();
        }
    }

    return categoryList;
}
```

*Note:* The function only returns categories with posts for the selection of the users.

Categories with no posts are not considered for selection.

**Figure 4.68**

*Categories Class*

```
public class Categories {  
    private Account account;  
    private String numberOfPosts;  
}
```

**Figure 4.69**

*Java Functions to Convert and Display Categories into Objects*

```
private void displayCategories() {  
    try {  
        for(Categories addedCategory : categoryList) {  
            FXMLLoader fxmlLoader = new FXMLLoader();  
            fxmlLoader.setLocation(getClass().getResource("../FXML/categories.fxml"));  
            VBox vbox = fxmlLoader.load();  
            categoriesController categoryController = fxmlLoader.getController();  
            categoryController.setData(addedCategory);  
            categoryContainer.getChildren().add(vbox);  
        }  
    } catch (IOException e) {  
        // TODO Auto-generated catch block  
        e.printStackTrace();  
    }  
}  
  
public List<Categories> getCategoryList() {  
    List<Document> categories = countPostsPerCategory();  
    List<Categories> ls = new ArrayList<Categories>();  
    for(Document category : categories) {  
        Categories addCategory;  
        addCategory = new Categories();  
        Account account = new Account();  
        account.setAccountID(category.getInteger("_id"));  
        account.setName(category.getString("CategoryName"));  
        addCategory.setAccount(account);  
        addCategory.setNumberOfPosts(category.getInteger("postCount") + " posts");  
        ls.add(addCategory);  
    }  
  
    return ls;  
}
```

*Note:* All categories are converted into VBox before being dynamically inserted into the category container.

**Figure 4.70**

*Java Function to Edit User's Preferred Categories*

```
public void selectCategories(int userID) {
    MongoClient mongoClient = null;
    try {
        mongoClient = MongoClient.create("mongodb://localhost:27017");
        MongoDB db = mongoClient.getDatabase("FacebookContentManagement");

        // Add the valid category IDs to the user's PreferredCategories array
        MongoCollection<Document> userCollection = db.getCollection("users");
        long result = userCollection.updateOne(
            Filters.eq("UserID", userID),
            Updates.set("PreferredCategories", loggedUser.getUserDetails().getList("PreferredCategories", Integer.class))
        ).getModifiedCount();

        if (result == 0) {
            System.out.println("Error: Failed to add CategoryIDs.");
            return;
        }
        System.out.println("CategoryIDs added successfully to UserID: " + userID);
    } catch (Exception e) {
        System.out.println("Error adding CategoryIDs to user: " + e.getMessage());
    }
}
```

*Note:* The current user's preferred categories field is updated into the new list of selected category IDs

## Chapter 5

### CONCLUSION

#### 5.1 Summary of Work Done

The system focused on improving Facebook's content generation to be more user-oriented by giving users control over the content they see. Thus, certain functionalities are adapted given their importance in the content of a user's newsfeed. Supplemented by a category-based feed, the system's functionalities involve the following:

*Category-based Feed.* Posts from all sources are now categorized into a specific category. Once a user selects several categories, the feed only returns posts under those preferences, excluding friendships, followed pages, and joined groups. If no categories are selected, the feed defaults back to posts from entities the user has connected to. For both cases, liked posts and reported entities are not included.

*User.* Users can create, login, and log out of their accounts. Furthermore, users can modify their respective information and preferred categories.

*Friends.* Users may send, accept, and delete friend requests to and from other users. Suggested friends remain based on the number of mutual friends with the user.

*Pages and Groups.* Page and group creators could modify the users involved in their communities and the posts under their timeline. Only the page creator can post on the page compared to all entities involved in the group. Non-creator members are allowed to leave the community whenever they want to

*Notifications.* Every interaction except reports made by the user sends a respective notification to different entities. Moreover, the User may mark the notification as read.

*Posts.* Given the category-based feed feature of the system, users create posts under a specific category. All posts may also be edited and deleted by the author, while non-authors could like and report all posts.

*Search.* The system implements a search function based on content and category for posts and names for pages, groups, and users. Only reported posts are not shown once searched by the user, allowing possible association with reported users, pages, and groups.

## **5.2 Challenges Faced**

Throughout the development phase of the system, the group faced issues, both technical and internal that incurred issues for the members:

*Understanding Facebook's Content Generation.* Given that the system covers how to improve content generation for Facebook, the group had to analyze in-depth the factors affecting content generated by the platform. Through the analysis, it has been observed that multiple factors are involved. Covering one aspect of it involved a multitude of logical functions just to meet the expected outcome.

*Unfamiliarity of Database used.* Transitioning from MySQL to MongoDB to meet the system's requirements proved to be tasking for the group due to unfamiliarity with the documentation. Much time was allotted to study the foundational concepts of NoSQL Databases to become familiar with the language, its syntax, and standard usage. This learning curve increased the level of difficulty in the design and implementation of the database and time allotment for its development.

*Unfamiliarity with JavaFX and SceneBuilder.* Designing the GUI with JavaFX and SceneBuilder involved manually incorporating functions and elements into the Frame. As

compared to Java Swing, JavaFX consists of multiple graphic components unfamiliar to the group. In-depth software learning utilized more time and resources than the whole design and implementation of the prototype frames.

*Project Management Issues.* Members had conflicting work ethics and calendar schedules that hindered specific phases of the development. As a result, certain group members had to adjust to cover for the lapses of the group produced by this challenge.

### **5.3 Lessons Learned**

After the system's completion, the group gained insights and knowledge about certain fields and areas:

*NoSQL Familiarization.* Implementing MongoDB allowed the group to acquire knowledge and skills in NoSQL designing and implementation, data insertion and retrieval, and how entities interact in NoSQL databases. A clearer vision of using scripts in MongoDB for creating and managing databases will be useful in future projects dealing with NoSQL databases.

*Front-end design through Java.* The system's development allowed the group to improve their front-end designing skills. Before the system's development, the members are only familiar with Java Swing components. Implementing JavaFX elements provided the developers a chance to explore more front-end designing.

*Duplication of Social Media Platforms.* Group members gained insights into how social media networks operate due to the system's nature. With the knowledge gained from this system, members could develop more social media apps.

## 5.4 Future Enhancements

Given the system's limited scope, future improvements can be considered to allow a wider range of audiences:

*Media Posts and Content.* Categorizing or adding context with media content such as photos and videos further expand the range of utilizing categories for the category-based feed. Moreover, allowing users to edit their profile pictures and cover photos give them the ability to beautify their user profiles.

*Connecting Entities based on Categories.* Given that each user selected categories of their preference, suggested friends and communities could be based on preferred categories, extending the functionalities of categories.

*Implementing the System into the Facebook Platform.* By implementing the logic behind the system into the Facebook Platform, all existing functionalities could compliment the proposed idea by the system.